



PDF Download
3725843.3756031.pdf
28 March 2026
Total Citations: 1
Total Downloads: 4358

 Latest updates: <https://dl.acm.org/doi/10.1145/3725843.3756031>

RESEARCH-ARTICLE

Symbiotic Task Scheduling and Data Prefetching

GILEAD POSLUNS, University of Toronto, Toronto, ON, Canada

MARK C. JEFFREY, University of Toronto, Toronto, ON, Canada

Open Access Support provided by:

University of Toronto

Published: 17 October 2025

[Citation in BibTeX format](#)

MICRO 2025: 58th IEEE/ACM
International Symposium on
Microarchitecture
October 18 - 22, 2025
Seoul, Korea

Conference Sponsors:
SIGMICRO

Symbiotic Task Scheduling and Data Prefetching

Gilead Posluns
University of Toronto
Toronto, Ontario, Canada
gil.posluns@mail.utoronto.ca

Mark C. Jeffrey
University of Toronto
Toronto, Ontario, Canada
mcj@ece.utoronto.ca

Abstract

Task-parallel programming models enable programmers to extract parallelism from irregular applications. Since software-based task-parallel runtimes impose crippling overheads on fine-grain tasks, architects have designed manycores with hardware support for task management. These hardware task-parallel systems can scale challenging workloads to hundreds of cores, but fail to use conventional prefetchers due to short (100-cycle) tasks. Lacking prefetching, they often expose DRAM latency to applications, fumbling the performance gains of hardware. We present the Task-Seeded Prefetcher (TSP) and Memory Response Task Scheduler (MRS), a symbiotic pair that boost performance in general-purpose task-parallel hardware. TSP learns and prefetches the data-access pattern of each task function, seeded with its descriptor that is queued by the task scheduler. MRS augments the baseline task-to-core dispatch policy by using prefetch status from TSP to optimize core utilization. Together, TSP and MRS provide speedups of up to $3.1\times$ (gmeans up to $1.4\times$) across 13 benchmarks on 256-core task-parallel systems that were already $3\text{--}60\times$ faster than parallel software.

CCS Concepts

• Computer systems organization → Multicore architectures.

Keywords

hardware task scheduling, data prefetching, task-level parallelism

ACM Reference Format:

Gilead Posluns and Mark C. Jeffrey. 2025. Symbiotic Task Scheduling and Data Prefetching. In *58th IEEE/ACM International Symposium on Microarchitecture (MICRO '25)*, October 18–22, 2025, Seoul, Republic of Korea. ACM, New York, NY, USA, 16 pages. <https://doi.org/10.1145/3725843.3756031>

1 Introduction

The arrival of manycore architectures presents programmers and architects with two key challenges: supplying hundreds of cores with (i) useful work to do and (ii) the data to work on. Unfortunately, apart from few celebrity application domains, having programmers, compilers, or runtimes productively express enough parallelism and accurately prefetch data has remained notoriously difficult. Programs that scale beyond a few cores remain exceptional.

The classic multithreaded programming model and its coarse-grain threads enables simple hardware but pushes the burden of work mapping and load balancing to the programmer. In contrast,

task-based programming models [11, 16, 46, 72] enable fine-grain parallelism and elevate abstraction to improve programmer productivity by pushing task scheduling decisions to the software runtime or hardware [43]. Tasks match the logical units of work in an application, such as processing a single graph vertex or running a single loop iteration. Moreover, task-level parallelism offers increased generality and flexibility over data- and vector-level parallelism, even supporting irregular parallelism [68] with its dynamically scheduled tasks and dynamically manifesting dependences.

Conventional manycores force *software task schedulers* to abstract away the multithreaded execution model, but require coarse-grain tasks of 100 K cycles to amortize their overheads [23, 37, 53]. *Hardware-assisted task schedulers* [28, 29, 38, 39, 44, 53, 63, 76, 78] can be $>100\times$ faster than software [69] by enabling fine-grain tasks of 10s to 100s of instructions. Hardware schedulers grant two benefits: (i) fine-grain tasks enable programmers to express the abundant parallelism demanded of general-purpose manycore systems and (ii) each fine-grained task has few accesses to shared memory.

Whereas task-parallel hardware gives cores work to do, hardware memory *prefetchers* strive to supply them with data by using past thread accesses to predict future ones. Commercial manycore prefetchers hide memory latency for regular striding and streaming access patterns [12, 22, 73, 81, 84] and more recently for irregular array-of-pointer patterns [30, 88, 94]. Research targets complex access patterns [13, 20, 41, 59, 82] or uses runahead to pre-execute a program's address generation code [49, 55–58]. These data prefetchers accelerate their target access patterns effectively, but depend on a long-running consistent access pattern.

Task-parallel application performance is highly sensitive to the cache locality induced by the task schedule [14, 38, 54]. Unfortunately, the hard-to-parallelize applications that benefit most from task parallelism tend to make many irregular memory accesses with terrible cache locality [82]. These applications are performance bottlenecked by memory latency rather than bandwidth [15, 82], so they can benefit from prefetching [58, 82, 94]. Unfortunately, no such prefetchers exist for hardware task-parallel systems.

Hardware task-parallel systems cannot use the prefetchers of the multithreaded execution model. Fine-grain tasks are too short for a conventional prefetch to be timely. Consequently, the prefetches must be triggered by the hardware scheduler in some way. Prior work has left addressing this need to future work [53] or put the prefetching burden on the programmer. Programmer-assisted prefetchers either permit only a single address as a prefetch hint [5, 86], or require the programmer to write separate prefetching tasks in addition to the compute tasks [95], akin to the inspector-executor programming model [75] or Helper Threads [45].

Accelerators with task-like models solve this problem for specific domains. Some fetch data for many independent tasks simultaneously [25, 34], working on each task as its data arrives. Others



This work is licensed under a Creative Commons Attribution 4.0 International License. *MICRO '25*, Seoul, Republic of Korea
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1573-0/25/10
<https://doi.org/10.1145/3725843.3756031>

prefetch necessary data while tasks are queued in PE input buffers, so the data will arrive before the task executes [32, 70, 71]. These accelerators hide memory latency in spite of short tasks because the memory accesses are built into the system at design time. Specialized hardware exploits task metadata to perform memory accesses before computation, providing domain-specific prefetching. Specialized schedulers [25, 54] further optimize locality within the target domain to maximize reuse of each datum loaded from DRAM.

Our insight is that the task metadata used for prefetching by specialized accelerators is *also available* to general-purpose task-parallel hardware systems. Therefore, these general-purpose systems can prefetch like specialized accelerators if they address three key requirements. First, task metadata is readily available in the task scheduler before tasks even run on cores, so these systems should learn task memory accesses based on task metadata. Second, the prefetcher must therefore integrate with the task scheduler to seed the prefetcher with upcoming (yet still idle) tasks. Third, the task scheduler should be aware of and leverage what data is available on chip to decide which tasks are granted core execution resources.

We propose the Task-Seeded Prefetcher (TSP) and Memory Response Task Scheduler (MRS), the first general-purpose hardware prefetcher capable of learning and prefetching all of a task's memory accesses and a set of prefetch-aware task scheduling policies that optimize the task schedule to further hide memory latency. TSP and MRS (i) learn the memory access pattern of every task in an application, (ii) prefetch the data a task will access before scheduling it for execution, and (iii) adjust the task schedule to exploit the prefetching. TSP and MRS neither change the task-parallel programming model nor require programmer or compiler annotations for prefetching. TSP is a general prefetcher applicable to any hardware task scheduler, and many different schedulers can be augmented with MRS's approach.

In short, we make the following contributions:

- A hardware memory prefetcher designed for hardware task schedulers that leverages idle task descriptor arguments to predict future memory addresses. It predicts addresses directly from task arguments, indirectly from loads seeded from task arguments, and striding accesses from task arguments.
- A hardware task scheduler that prioritizes tasks whose prefetches filled over those that will stall a core.
- Integration of these techniques into general-purpose manycores with hardware-assisted task scheduling.
- An evaluation on 256-core manycores with four distinct scheduling policies based on prior work, showing speedups of up to $3.1\times$ (gmeans up to $1.4\times$) over the baselines.

2 Background and Motivation

Manycores with architectural support for tasks extract far more parallelism than those without [39, 44, 76, 78], scaling even hard-to-parallelize irregular applications to hundreds of cores [28, 38, 69]. They tend to use small, in-order cores [28, 29, 38, 40, 44, 69, 76, 78], trading more power-efficient task-level parallelism for less instruction-level parallelism. While task-parallel architectures effectively scale up compute, they continue to stall out on irregular memory accesses.

```

1 void SSSPTask(Vertex* v, int dist) {
2   if (dist > v->dist) return;
3   for (Edge* e = v->nbegin; e != v->nend; ++e)
4     if (dist + e->dist < e->v->dist) {
5       e->v->dist = dist + e->dist;
6       runtime::enqueue(SSSPTask, e->v, e->v->dist);
7     }
8 }

```

Listing 1: Task function for Dijkstra's SSSP algorithm.

The algorithms for the single-source shortest paths problem (sssp) aptly illustrate this tradeoff. Listing 1 shows an sssp task function for Dijkstra's algorithm that targets a hardware task-parallel system. Hardware provides an enqueue instruction wrapped by a light software runtime through which tasks create new children tasks. The hardware scheduler queues these tasks and decides when and where to *dequeue* tasks to cores. Hardware scheduling policies may opt for priority ordered dispatch [39, 78] (preferred by sssp), optimize locality for unordered algorithms [54], or maintain an explicitly declared dependence ordering [29, 53, 83]. We motivate our work by characterizing key properties of the sssp task and other task-parallel benchmark applications (methodology in Sec. 5.1). We compare a hardware task-parallel system with three levels of cache and a speculative spatial task scheduler [38] to a variant with an *ideal* memory system that always hits in the L2 cache, ignoring the actual latency data takes to arrive. We find that task-parallel programs like sssp have the following three properties.

Task-parallel programs are limited by memory latency. The sssp task in Listing 1 consists almost entirely of chains of dependent memory accesses. The loop bounds ($v \rightarrow nbegin/nend$) are loaded from the vertex argument and each loop iteration waits on a chain of three dependent loads ($e \rightarrow v \rightarrow dist$). Fig. 1a shows that these dependent load chains expose significant memory latency to the core, causing an IPC of only 0.4 for sssp and at most 0.6 for other benchmarks. The ideal system eliminates memory latency beyond

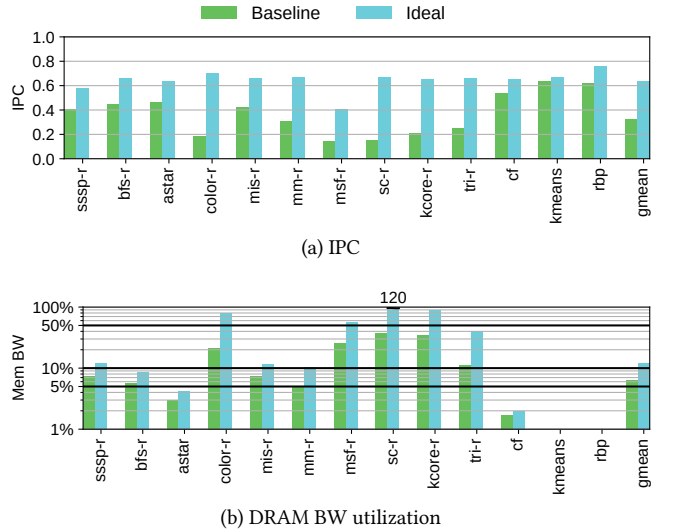


Figure 1: Comparing a non-prefetching baseline vs. ideal (100% L2 hit rate) system on (a) IPC and (b) DRAM BW.

the L2 and grants significant IPC increases for all benchmarks except kmeans: $> 2\times$ for color, mm, sc, kcore, and tri.

Fig. 1b reports DRAM bandwidth (BW). We provision memory controllers similarly to prior work on bandwidth-limited prefetching [66] or infinitely¹ for the ideal system. Our most memory-heavy benchmark (sc) consumes only 37% of DRAM BW. With a perfect L2 hit rate, average BW more than doubles, but only idealized sc can saturate the memory controllers. Our system features many small, in-order cores, so benchmarks like sssp have only one in-flight load per core at a time. With an 11-cycle L2 hit latency, these applications do not fully utilize their available memory bandwidth, even with perfect prefetching. These measurements indicate that, even with hardware-assisted task scheduling, memory latency causes significant core idling yet these workloads face little risk of exhausting memory bandwidth if memory latency were eliminated.

Tasks are too short for conventional prefetching. Listing 1 is a qualitatively short segment of code that visits one vertex and conditionally creates new tasks to visit its neighbors. On some inputs, it runs fewer than 100 instructions 90% of the time [39]. Many dynamic instances of the task will return immediately at line 2 [69]. Of the instances that reach the loop, software-side task-parallel innovations like enqueue trees [77, 93] parallelize the loop iterations across several tasks, with each task only running a few iterations.² Table 1 reports the average time between task dequeue and completion on the idealized system: 8/13 benchmarks have average task lengths < 200 cycles, less than the > 200 -cycle latency to memory [50]. Consequently, by the time a core starts running a task, it is already too late to hide memory latency by prefetching.

Hardware task schedulers lack the striding access pattern that conventional prefetchers rely on to target irregular algorithms [58, 94]. In fact, they lack any access pattern at all. A hardware task scheduler pops each task using a special instruction rather than a load. The task queue is a black box with its contents hidden from both software and conventional cores. Fig. 2 (bottom) illustrates how a hardware task scheduler interferes with conventional prefetchers. Even though the loads of a task are predictable, they depend on an unpredictable task pop. Lacking foreknowledge of which task will be popped, a prefetcher can only predict the task's loads after it pops, which is too tardy to be timely.

On one hand, hardware task schedulers make the task schedule flexible and eliminate software scheduling overheads, enabling fine-grained tasks to maximize parallelism [3, 38, 63]. On the other hand, they preclude conventional prefetchers. Consequently, prior work on hardware task parallelism often forgoes conventional prefetchers [3, 38–40, 53, 63, 69, 78, 95]. Unlike a software schedule, there is no static task array in which to look ahead (Fig. 2 top). A prefetcher for a hardware task-parallel system cannot be triggered by program execution. Unlike conventional prefetchers, a task-parallel prefetcher should be controlled by the task scheduler.

Task memory accesses are predictable. All deterministic memory addresses accessed by a task are determined by some combination of the task's arguments and its prior memory accesses.

¹We measure the memory BW of the ideal system as if L2 misses reached memory, even though the data teleports into the ideal L2 as soon as the request is sent to the LLC, so the core can continue execution as if it hit in the L2.

²Enqueue trees enable tasks to remain fine-grained as inputs grow, meaning that larger inputs will have more tasks, but not longer tasks.

Table 1: Task length and memory characteristics.

		bfs-r	sssp-r	astar	mis-r	mm-r	msf-r	color-r	sc-r	kcore-r	tri-r	cf	kmeans	rbp
# static task funcs		3	3	1	6	9	13	9	14	6	30	14	16	3
Ideal avg. task cycles		37	47	857	61	98	340	154	166	117	116	612	4.5k	854
Avg. dyn. loads		2	5	15	3	4	12	7	10	9	6	28	141	48
Predictable loads (%)		93	99.9	86	99.9	94	44	85	90	93	95	97	99	79
Avg. dyn. cache lines		1	4	7	2	3	3	4	4	4	3	6	36	18
Predictable lines (%)		99	99.9	89	99.9	95	86	85	97	98	94	95	99	79

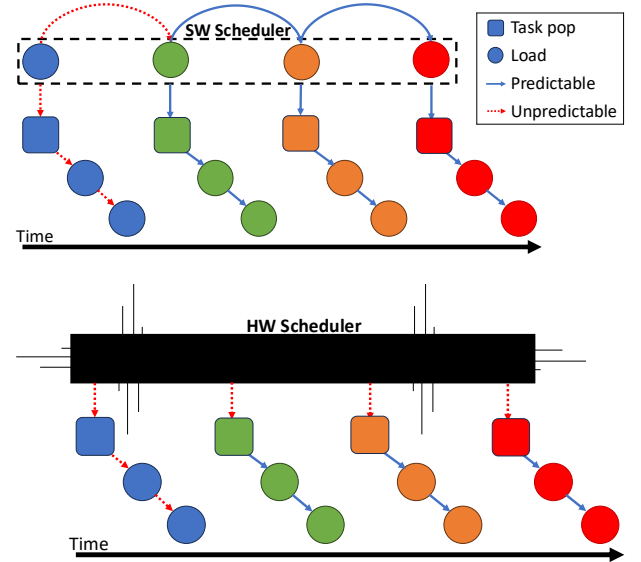


Figure 2: Prefetchability of task memory accesses by a conventional prefetcher with software vs. hardware schedulers.

For instance, Listing 1's memory accesses form a chain starting from its vertex pointer argument. The first few access addresses ($v \rightarrow \text{dist}/\text{nbegin}/\text{nend}$) are equal to the first task argument plus a constant offset. The next set of accesses ($e \rightarrow v/\text{dist}$) depend on the value returned by the load to $v \rightarrow \text{nbegin}$ in the same way, and this pattern continues for the entire task function.

We observe that these addresses can be predicted *before* running the task. We adapt the approach of the Indirect Memory Prefetcher (IMP) [94]. IMP detects a striding access pattern in a thread, then detects if that data is used for address generation. Task-parallel hardware is not dependent on a fixed sequence of tasks, so no initial stride exists to trigger IMP. Instead, task-parallel hardware provides a better alternative: rather than *detect* all prefetch-critical values, the hardware scheduler *seeds* task arguments as critical values to compute prefetch addresses. Like IMP, we consider an address to be predictable if it is an affine function of a prior memory access. Unlike IMP, we seed prefetchable addresses with the task arguments

that are readily available in hardware task-parallel systems. In other words, an address can be predicted easily if it can be expressed as:

$$\text{address} = \text{scale} \cdot [\text{ref}] + \text{offset} \quad (1)$$

where *scale*/*offset* are constants and *ref* is either (i) a task argument or (ii) a value returned by a prior predictable load.

This approach may miss some deterministic accesses but Eq. 1 is simple and cheaply implementable by a shifter and adder [94] (for power-of-2 *scale*). More expensive approaches, like runahead execution, would predict all deterministic accesses. Table 1 shows the effectiveness of Eq. 1 at predicting memory accesses across our benchmarks. The bottom two sets of rows show the average number of dynamic loads and the average number of dynamic cache lines accessed by those loads, respectively (excluding task-private stack accesses). They also show the percentage of those loads and lines that can be predicted using Eq. 1, (i.e., counting the accesses for which TSP is able to issue prefetches). For example, *astar* tasks issue 15 dynamic load instructions each, on average, covering 7 cache lines. TSP found values for *scale*, *offset*, and *ref* in Eq. 1 that predicted 86% of these loads, covering 89% of cache lines accessed. We observe that 12/13 benchmarks access multiple cache lines per task. Consequently, prior work that adds prefetch hints to task descriptors [5, 86] is not sufficiently general: task descriptors must be fixed size to hold in hardware task queues, so would not support a variable number of hints.

Inspecting the assembly of our benchmarks, we find that predictable loads access vertex data and edge lists. Sophisticated conventional prefetchers [54, 58, 82, 94], also target this data because it has poor locality without prefetching. This suggests that our method for predicting load addresses identifies and predicts the same loads targeted by specialized prefetching techniques.

Hardware task schedulers will benefit from prefetching. This prefetching requires scheduler support. Prefetching and task scheduling are powerful on their own, but are stronger working together.

3 TSP and MRS Overview

Fig. 3 shows a representative manycore augmented with hardware-assisted task scheduling [39, 78]. Each tile has a group of cores, with private L1 caches, an L2 shared among them, and a slice of a fully-shared L3. The tile's *task unit* queues tasks and schedules them to cores. The *task queue* (TQ) and *task send buffer* (TSB) hold *task descriptors*, which consist of a function pointer, arguments, and any other metadata required by the *task scheduler*, like priority.

Abstractly, a running task *enqueues* new tasks to an abstract global scheduler using an enqueue instruction, while an idle core *dequeues* a task for execution. Concretely, a core executing an enqueue instruction adds a task descriptor to its local TSB, which buffers it to decouple enqueues from communication [40, 78]. The TSB asynchronously sends the task to some task unit which the local hardware scheduler selects based on the task mapping portion of its scheduling policy. Upon arrival at its destination task unit, the task is placed into the local TQ. A core executing a dequeue instruction receives a task descriptor from the TQ, selected by the local scheduler according to its scheduling policy (e.g., local priority order) then runs the descriptor's task function with its arguments provided through registers. The scheduler chooses which cores receive which tasks, and in which order. Some task scheduling policies

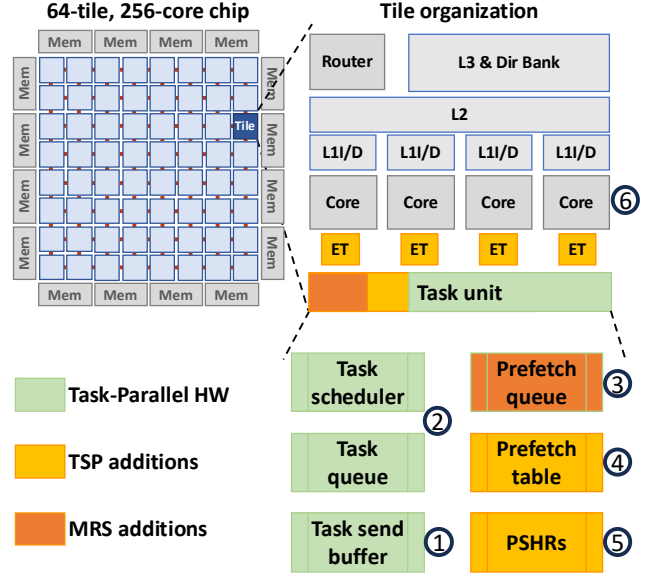


Figure 3: Full system architecture with TSP/MRS.

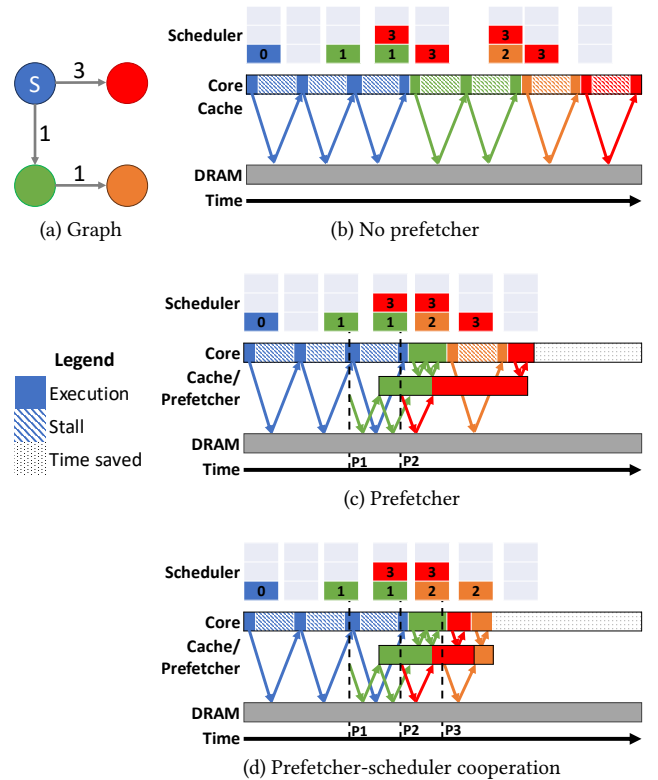


Figure 4: Schedules for executing task-based sssp.

map tasks to tiles in a way that optimizes cache locality [38], while others optimize priority drift and NoC traffic [78].

Task-parallel schedulers for ordered algorithms (e.g., Listing 1) typically select a high-priority task (low distance from source) to execute at each dequeue. Fig. 4b shows a short execution of sssp on an illustrative single-core system for the small graph in Fig. 4a. The scheduler initially holds one task for the blue vertex, which accesses memory three times in series. First, the task loads blue vertex data, then the data of the green and red vertices, for which it enqueues tasks. The scheduler selects the green vertex to process next, and the green vertex task enqueues the orange vertex. For this example, we presume no cache locality among tasks (a real system would not run all these tasks on a single core), so the green vertex task must access memory twice for its own data and for the data of its orange descendant. The scheduler selects the orange vertex to run next, and finally the red vertex. Most core time is spent waiting on cache misses. These are compulsory misses to randomly accessed data—only a prefetcher can convert these misses into hits.

3.1 Prefetching with a task-parallel scheduler

A conventional prefetcher can do little to help task-parallel sssp. When the core processes the blue vertex, a conventional prefetcher cannot anticipate that the green one will be next. On the other hand, the scheduler does know which task it will select next, and can seed an integrated prefetcher with that information. Fig. 4c shows the execution schedule of sssp with TSP contributing prefetches. At time P1, the scheduler seeds the prefetcher with the task descriptor for the green vertex, allowing it to prefetch the data that the green vertex will access. At time P2, the scheduler seeds the prefetcher with the red vertex. This adds memory-level parallelism (MLP) by allowing the DRAM accesses of the green vertex to overlap with the blue vertex using the core, and the DRAM access of the red vertex to overlap with the green vertex using the core. This enables the tasks processing the green and red vertices to hit in the cache when they are on the core, saving time. Although loads for the green and red vertices hit, the orange vertex still misses in the cache because it was selected to run by the scheduler immediately after being enqueued, so the scheduler never seeded the prefetcher with it.

The orange vertex misses because the scheduler selected it to be processed right after receiving its enqueue. There was no time to prefetch for it, even though the scheduler was seeding the prefetcher. However, a task-parallel scheduler has *flexibility*: there is nothing wrong with visiting the red vertex before the orange one, even though the orange one comes first in the priority order. For example, concurrent priority schedulers often relax (reorder) priority dispatch order to reduce contention [9, 25, 78, 96]; correctness is not lost when adjusting the task schedule to better take advantage of prefetching. Even a FIFO or random ordered scheduler would be correct for sssp, although it would lose work efficiency due to executing many useless tasks [69]. Fig. 4d shows the execution schedule with both TSP and MRS providing prefetcher-scheduler coordination. By choosing to visit the red vertex with ready data before the higher-priority orange one, the prefetcher-aware scheduler gains the time for the orange vertex's data to be prefetched, and seeds the prefetcher with the orange vertex at time P3. The orange vertex finished processing around the same time in both Fig. 4c and Fig. 4d, but with a prefetcher-aware scheduling policy the red vertex could be processed while orange waited for its data.

3.2 Predicting task accesses with TSP

Listing 1 contains three types of loads, all of which are predictable from its arguments. Although other load patterns exist [10] (e.g., `a[hash(x)]`), these three cover most loads in our benchmarks.

Argument-based loads like `v->dist` are affine functions of task arguments. TSP predicts them by applying Eq. 1 to the task descriptor arguments, which are stored and managed by the task scheduler.

Indirect loads like `e->v->dist` depend on prior loads. Loads of the form `a[b[i]]` cannot be predicted directly using the task descriptor, but they can be predicted using the values returned by prior prefetches. IMP [94] and DMP [30] detect and prefetch multi-level indirect memory access patterns by using the stream of prefetches for a prior pattern to seed the next level of prefetching. Similarly, TSP predicts indirect loads in a task using the values returned by prior prefetches for that task. TSP's indirect prefetching is recursive, with depth limited only by available prefetching resources.

Inner-loop loads like `e->dist` stride across memory addresses in a loop. A stride or stream prefetcher could predict these accesses if the loop ran for many iterations, but for a short task, prefetching must be performed before the task starts running. To prefetch an inner loop, TSP must predict not only the base address and stride of the access pattern, but also the loop bounds [58]. Strides are typically constants for each load so TSP can learn them just like a stride prefetcher. Loop bounds are often data-dependent, so TSP predicts them just like it predicts addresses. Once all three loop parameters (stride, base address, and bounds) are learned, TSP issues all prefetches for the inner loop simultaneously.

In a task-parallel system, schedulers and prefetchers can help each other. Schedulers should tell prefetchers what to prefetch. Prefetched data can guide schedulers toward faster schedules.

4 Implementation

Fig. 3 shows how we augment each task unit in a tiled manycore with TSP, our prefetcher that learns task memory accesses and prefetches them when seeded with a task descriptor. We also augment the task scheduler with MRS, our scheduling policy that reactively adjusts a baseline task schedule to better take advantage of TSP.

4.1 MRS implementation

MRS adds one structure—a *prefetch queue* (PQ)—to each task unit. The PQ duplicates the data structure used by the task scheduler to select the next task for execution. In this case, that is an ordered queue, but a simpler scheduler could instead use a ring buffer or stack. When the baseline task scheduler adds a task to its TQ, MRS also adds it to the PQ, which uses the same ordering heuristic. Whereas the baseline scheduler accesses its order queue in response to dequeue requests from cores, MRS proactively seeds TSP with task descriptors from its PQ as TSP becomes ready to perform them.

The interval from when TSP begins prefetching a task's memory accesses to when those prefetches fill varies based on the position of the data in the memory hierarchy. Starting a task's execution while its prefetches are pending exposes the remaining memory latency to the core. Most tasks in a task-parallel program are independent [39], so it is acceptable to reorder them. TSP enables MRS by notifying the scheduler when prefetching for a task completes. When the task scheduler reacts to a dequeue instruction, MRS skips

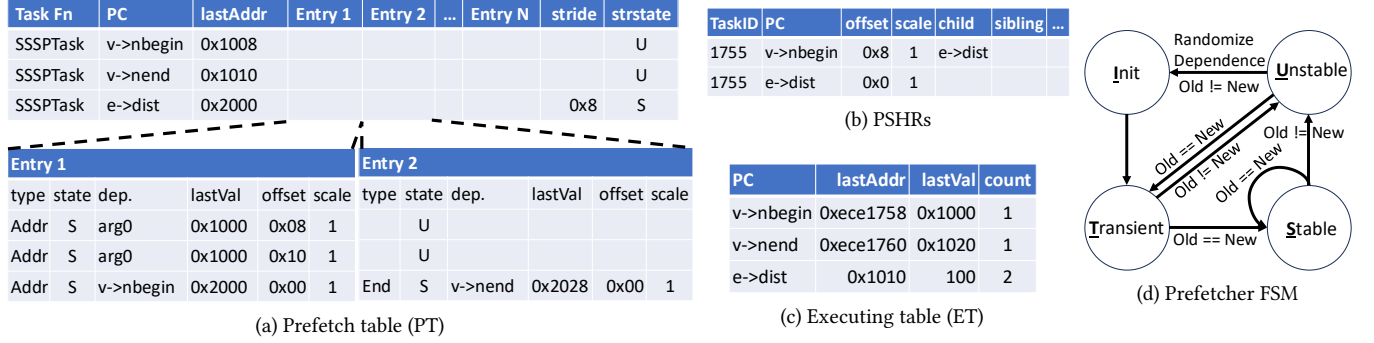


Figure 5: TSP structures and finite state machine with example training on Listing 1.

those task descriptors that are eligible for prefetching but have not yet had their memory accesses arrive in the cache. Instead, MRS dequeues the highest-priority task (according to the baseline scheduling policy) that has its memory accesses already prefetched, or which is not eligible for prefetching. If no such tasks exist, MRS falls back to the baseline scheduling policy to keep the core busy, dequeuing a task that will observe some memory latency.

4.2 TSP implementation

TSP prefetches data for all of a task's predictable loads into the L2 cache. TSP initiates prefetching when the task scheduler seeds it with a specific task descriptor that has not yet been dispatched to a core. TSP requires the task descriptor to contain a task function, arguments, and unique identifier. The TSP per-tile *prefetch table* (PT) holds mappings of (task function, load PC) pairs to prefetcher entries. Fig. 5a shows trained rows and entries for SSSPTask from Listing 1. TSP prefetches the memory accesses of that task function in the following stages.

Argument-based prefetching: The load to v->nbegin in Listing 1 is predictable using argument 0 (v). Entry 1 in v->nbegin's row of Fig. 5a holds this relationship. When seeded with a task descriptor, TSP searches all rows in its PT that match the descriptor's task function. For each matching row with a STABLE entry that depends on an argument, TSP calculates the prefetch address as in Eq. 1 but with the argument value substituted for *ref*, then issues the prefetch. For example, Entry 1 for Rows 1 and 2 in the PT hold STABLE loads that depend on arg0. TSP allocates a *prefetch status handling register* (PSHR), depicted in Fig. 5b, for each prefetch it issues. The PSHR TaskID refers to the specific task descriptor that this prefetch is for. If TSP cannot allocate enough PSHRs, it pauses prefetching and applies backpressure to MRS until enough PSHRs become available for it to receive more task descriptors.

Indirect prefetching: The first load to e->dist in Listing 1 cannot be predicted based on the arguments, but may be predicted based on the value prefetched for v->nbegin. Entry 1 in e->dist's row of Fig. 5a holds this relationship. When TSP finishes argument-based prefetching, it searches its selected PT rows for STABLE entries that depend on a load for which a PSHR exists with the same TaskID. Given a PSHR match, TSP allocates a new PSHR for the dependent load, but does not immediately issue a prefetch. When the prefetch for the dependence fills, TSP uses the loaded value to calculate the

prefetch address for each dependent PSHR, and performs the prefetches at that time. TSP continues allocating dependent PSHRs until no predictable loads remain for the given task function, allowing dependence chains of unlimited depth. PSHR dependences are tracked using child and sibling pointers, avoiding associative lookups when prefetches fill.

Inner-loop prefetching: Loads to e->dist in Listing 1 after the first follow a striding pattern up until they reach the address prefetched for v->nend. Entry 2 in e->dist's row of Fig. 5a holds the loop-end relationship, while the row's stride and strstate fields store the stride similarly to a conventional stride prefetcher [12]. In addition to loop-end addresses, TSP has a third type of entry, Count, which holds a loop tripcount instead of an end address.

When TSP allocates a PSHR for a load like e->dist, which has a STABLE stride and a STABLE loop bound entry, TSP issues prefetches starting from the initial prefetch address, then incrementing by the stride, up to the bound. If the base address and bound have different non-argument dependences (e.g., v->nbegin and v->nend) TSP holds both dependence values in the PSHR and initiates inner loop prefetching once both are present. PSHRs that depend on a striding load (e.g., e->v->dist) will trigger an indirect prefetch for each value retrieved by the striding load.

4.3 Training TSP

TSP trains its PT entries on each memory access, learning the dependence, offset, and scale fields for address entries. Each row in the PT holds the last address used to train that load PC, and each entry holds the last value of its dependence. TSP also maintains an *executing table* (ET) per currently executing task (Fig. 5c), which maps load PCs to (address, value, count) tuples.

When TSP observes a memory access, it updates the ET entry for that load PC with the new address and value, and increments count. If this access was the first occurrence of that load PC within the task, TSP also updates all Address-type PT entries for that load PC using their stored lastAddr and lastVal (Fig. 5a) and the new address and value. TSP updates the offset and scale values according to the formulae:

$$newScale = \frac{oldLastAddr - newLastAddr}{oldLastVal - newLastVal} \quad (2)$$

$$newOffset = newAddr - newScale \cdot newLastVal \quad (3)$$

In these equations, *oldLastAddr* represents the address of the last time TSP saw this load PC, whereas *newLastAddr* represents the address being accessed this time. *oldLastVal* represents the value of the dependence when *oldLastAddr* was accessed, while *newLastVal* is the current value. For argument dependences, *newLastVal* is an argument value from the task descriptor. For indirect dependences, it is read from the dependence’s ET entry. These equations first use the ratio of the changes in the dependence value and address to determine the slope of the affine function from Eq. 1, then use this new slope to compute the new intercept.

TSP trains inner-loop End/Count-type entries each time a task finishes execution. End/Count-type entries are updated identically to Address-type entries, reading the ET lastAddr or count fields, respectively, to provide their *newLastAddr* value.

The FSM of Fig. 5d determines whether and how the Entry state updates, based on new values of offset and scale. If this access was not the first occurrence, TSP updates the PT’s stride field according to $newStride = newAddr - lastAddr$, and updates *strstate* using the same FSM without the U→I transition.

Each row in TSP’s PT contains multiple entries (e.g., 8), which may be fewer than the number of possible dependences. When a TSP entry observes multiple consecutive changes in its offset and scale (what would otherwise be a U→U transition), it randomly selects a new dependence from either the task arguments or the set of valid load PCs in the ET, and returns to INITIAL state. This allows TSP to search for dependences across all arguments and prior loads with finite-length PT rows.

4.4 Task lifetimes with TSP and MRS

The numbers in Fig. 3 indicate the life-cycle of a task. Task descriptors maintain the same Core→TSB→TQ→Core flow as prior hardware task-parallel systems, however MRS also inserts the task into the local PQ at the same time that it inserts it into the TQ. The PQ uses the same ordering policy as the TQ but holds fewer task descriptors (when MRS removes a task from the TQ for any reason, it also removes it from the PQ, but not vice versa). When a task reaches the head of the PQ, MRS seeds TSP with it. TSP uses its PT to determine which data to prefetch for the task, then allocates it PSHRs. Once the task’s final PSHR is freed, MRS prioritizes the task for dequeue and soon dispatches it to a core to execute.

4.5 Mitigating harmful prefetches

Certain access patterns, such as $a[hash(x)]$ or complex control flows, could “trick” TSP into briefly learning an incorrect offset and scale. TSP would issue useless prefetches that pollute the cache. TSP mitigates this problem with a usefulness counter in every PT row. TSP increments this counter every time a row entry confirms its prediction (the S→S transition in Fig. 5d). TSP decrements the counter by 128 every time a prediction turns out to be wrong (the S→U transition). TSP does not prefetch rows with negative usefulness counters, enabling it to avoid inaccurate prefetches. The usefulness counter could be used to penalize other scenarios, such as prefetches for contended cache lines that tend to be invalidated before being accessed. However, we found that our benchmarks perform better without penalizing invalidations.

Table 2: TSP and MRS estimated area and energy overheads.

	Entries	Entry size (B)	Size (KB)	Area (mm ²)	Energy (nJ/access)
PSHRs	64	47	3	0.012	0.09
ET	4×128	25	12.5	0.028	0.005
PQ(TCAM)	256	2×8	4	0.175	0.070
PT(CAM)	256	10	2.5	0.013	0.020
PT	256	$22 + 8 \times 26$	57.5	0.094	0.042

TSP can also hurt performance if its prefetches saturate DRAM or NoC bandwidth. TSP mitigates this scenario by disabling some of its PSHRs when prefetch filling begins to take much longer than memory latency, and slowly re-enabling them over time.

4.6 TSP and MRS hardware overheads

We measure the hardware overheads of TSP and MRS using CACTI [85] with 32nm technology and base our TCAM area and power estimates on Nii et al. [61]. The PT is split into CAM and indexed portions, similarly to a cache with tags and data. Table 2 shows the hardware overheads of each structure. Overall, the area overhead of TSP and MRS is 13.5% of a 1MB L2 cache. This overhead is significant, but we find that TSP and MRS are often more valuable than a large cache (Sec. 5.2). The TSP PT and MRS PQ dominate energy cost, but are accessed only once per task enqueue/dequeue (e.g. with 100-cycle tasks, two searches every 25 cycles).

To put these overheads in context, a conventional approach to spend area/power to extract MLP from irregular fine-grained tasks is hardware multithreading [4]. Davis et al. found that every additional HW thread context adds 5–6% area to a simple core [26]. In our modeled system (Table 3), the L2 caches contribute 16.5% of the total cache area, with the bulk of that area going to the LLC. Together, TSP and MRS add around 2% cache area to the chip. With a 75/25 cache/core split, this is equivalent to using two hardware threads per core, for at most $2 \times$ MLP. With 128 PSHRs, TSP can add far more than $2 \times$ MLP.

5 Evaluation

We evaluate TSP and MRS across four separate task scheduling policies and two system sizes, using 13 task-parallel algorithms (22 benchmark-input pairs). We find that TSP and MRS consistently improve application performance, by margins of up to $3.1 \times$ over prior task-parallel hardware.

5.1 Experimental methodology

Modeled systems: We adapt the open-source, cycle-level, execution-driven Swarm [38, 39] simulator based on Pin [51, 65] to model hardware task-parallel manycores, shown in Fig. 3, with parameters in Table 3 that are consistent with prior work [69, 93]. We use detailed core, cache, NoC, and memory models, including NoC contention and limited DRAM BW. The baseline scheduling policies use task priority order [68] to dispatch tasks to cores. We evaluate TSP and MRS across four task scheduling policies by varying two design parameters: synchronization and task mapping.

Our schedulers use either *speculative* or *non-speculative* synchronization. Speculative schedulers like Swarm [39] provide the

Table 3: System Configurations.

Cores	256 cores in 64 tiles (4 cores/tile), 2 GHz, x86-64 ISA; single-issue in-order, scoreboarded (stall-on-use) [38, 40]
L1 caches	32/2 KB, per-core, split D/I, 8-way, 2-cycle latency
L2 caches	1024/64 KB, per-tile, 8-way, inclusive, 9-cycle latency
L3 cache	256/16 MB, shared, static NUCA [42] (4096/256 KB bank/tile), 16-way, inclusive, 12-cycle bank latency
Coherence	MESI, 64 B lines, in-cache directories
NoC	Four 8×8 meshes, 192-bit links, X-Y routing, 1 cycle/hop when going straight, 2 cycles on turns (like Tile64 [90])
Main mem	16 controllers at chip edges (1 controller/16c), 120-cycle minimum latency, 48GBps BW/controller [66]
Queues	64 TQ and 24 TSB entries/core (16384 and 6144 total)
Spills	Spill 15 tasks when the task queue is 85% full
Conflicts (speculative schedulers only [39])	2Kbit 8-way Bloom filters, H_3 hash functions [19] Tile checks take 5 cycles (Bloom filters) + 1 cycle per timestamp compared in the commit queue 16 commit queue entries/core (4096 total)
Random map	Send tasks to tiles uniform randomly
Spatial map	Statically hash programmer-provided spatial hints to tiles [38]

illusion of atomic or sequential execution by tracking task read/write sets and aborting tasks that violate data dependence order. Speculative task-parallel programs do not need software synchronization. Speculation finds parallelism in applications that require strict task order [69]. We simulate all task and speculation overheads (e.g., task traffic, running misspeculating tasks until they abort, simulating conflict check and rollback delays and traffic, etc.). In contrast, non-speculative priority schedulers [78] require software to synchronize data dependences with locks or lock-free techniques. Non-speculative schedulers demand lower hardware overheads than speculative, making them attractive for applications that tolerate task reordering [78]. We model these schedulers by disabling all speculation mechanisms and modifying our benchmarks to synchronize with compare-and-swaps. For non-speculative scheduling of applications that do not tolerate task reordering but have bulk-synchronous parallelism available per priority, we also disable out-of-order execution of tasks [40].

Our schedulers map tasks to tiles either randomly or using Spatial Hints [38]. The latter is a technique that allows the programmer to convey locality to hardware, to run tasks likely to access the same data in the same place. When a new task is created, software provides an integer *hint* that abstractly denotes the data that the task is likely to access. The hardware scheduler sends tasks with the same hint to the same tile to localize data accesses. The drawbacks of hints are that (i) they can cause load imbalance when some objects receive many more tasks than others, (ii) they require programmer annotation, and (iii) there is at most one hint per task, so tasks that access multiple objects benefit less or not at all.

Testing each combination of synchronization and mapping policies results in four separate schedulers, all of which are represented in prior work: SPECRAND is similar to the original Swarm [39], while SPECSPAT resembles Swarm with Spatial Hints [38]. NONSPECRAND is an example of a RELD [78, 92] system, while NONSPECSPAT uses hint-based task mapping without speculation, similarly to Dalorex [63] or PolyGraph in asynchronous mode [25].

We test two cache sizes across the four schedulers. Our benchmark inputs are modestly sized (Table 5), but much larger inputs

exist. We cannot feasibly simulate large inputs due to memory and simulator run time constraints, so we emulate the effect of a larger working set by cutting the cache size. Our smaller system shrinks all cache sizes by a factor of 16, producing a graph-size-to-cache-size ratio similar to the ratio that would be observed with the use of Twitter [48] as an input. For random memory accesses, the ratio of working set size to cache size will determine the cache hit rate. **Prefetchers:** For each system, we test four prefetchers. Our baseline (B) is the scheduler without any prefetching, matching prior work on hardware-assisted task scheduling [3, 38–40, 53, 63, 69, 78, 95]. We verified that adding a simple LLC-level next-block prefetcher to this system has no impact on performance. We also test a system with an optimistic implementation of IMP at the L1 caches of each core (I). We then test a system that implements TSP and MRS, but disables training. Instead, we allow one entry of one row of TSP per task function to be manually programmed by the programmer, and add these prefetches to our benchmarks where possible. This one-address-per-task prefetching matches Tesseract’s [5] prefetching for RPCs (T). Next, we remove the restriction to a single programmed prefetch and program TSP with as many prefetches as could be statically determined from the application source code, including indirect and loop prefetches (P). Finally, we enable training, and test our full system with TSP and MRS (M).

Benchmarks: Table 4 lists our benchmark applications, covering domains including graph algorithms, statistical inference, numerical optimization, and machine learning. We draw these benchmarks from prior work that uses the Swarm simulator, including every benchmark used to evaluate Hive [69] as well as the locales-based benchmarks from Espresso [40]. Prior work found these benchmarks are 3× (rbp) to 60× (sssp) faster on task-parallel hardware

Table 4: Benchmark abbreviations, sources, inputs, and non-speculative synchronization strategy.

Abbrv.	Inputs	Non-Spec	Description
bfs [47]	r, o	Relaxed	Finds the smallest number of edges to reach each vertex from a source
sssp [68]	r, o	Relaxed	Finds shortest paths from a source to all vertices (non-neg. weights)
astar [39]	g	Relaxed	Finds the geographic shortest path from source to target
mis [80]	r, o	None	Maximal independent set selects a set of non-neighboring vertices such that no additional vertices may be added
mm [80]	r, o	None	Maximal matching selects a set of edges such that no two share an endpoint and no additional edges may be added
msf [80]	r, o	None	Kruskal’s algorithm to find the minimal spanning forest of a graph.
color [36]	r, o	None	Assigns each vertex a color such that no neighbors share a color, largest degree first
sc [27]	r, o	Relaxed	Greedy selects sets to approximate the smallest cover of all elements
kcore [27]	r, o	Synchronous	Finds the subgraphs with minimum degree k ($\forall k$) by peeling the graph
tri [79]	r, o	Synchronous	Graph triangle counting
cf [79]	r, o	Synchronous	One iteration of collaborative filtering to train a recommendation model
kmeans [52]	rand	Synchronous	Unsupervised machine learning that finds k (=40) clusters in unlabeled data
rbp [8]	i	Relaxed	Residual belief propagation runs statistical inference on Markov Random Fields

Table 5: Input datasets and their vertex counts, edge counts, and weight types or other detail.

	Input	V	E	Weights
r	USA roads [1]	24 M	58 M	Distances (32-bit)
o	com-Orkut [48, 91]	3 M	117 M	Random 1-256 (32-bit)
g	Germany roads [2]	12 M	32 M	Distances (64-bit)
rand	Random 32-D [52]	4 M	–	16 generating clusters
i	Ising	200 × 200	80 K	–
m	MovieLens 25M [35]	370 K	50 M	25 M ratings, 1 M tags, 62 K movies, 162 K users

than multithreaded software [69]. We exclude *des* and *nocsim* from Espresso because their implementations rely on virtual functions in a way that conceals task function pointers from the hardware. We exclude *genome* due to limited parallelism at hundreds of cores [40].

Table 5 lists our inputs. For benchmarks that can accept multiple inputs, we test both, and use their abbreviation to differentiate. For our analysis, we treat the one benchmark with two different inputs as two separate benchmarks, for 22 total benchmarks instead of 13.

5.2 Performance

Fig. 6 shows the performance impact of adding progressively sophisticated prefetchers to each of our 8 systems (4 schedulers × 2 caches). Each group of bars shows a benchmark, with height representing execution time relative to the baseline (**B**) with full-sized cache. Numbers represent the baseline run time in cycles. Each bar breaks down how cores spend their cycles, executing (i) non-speculative tasks or speculative tasks that eventually commit or (ii) later abort; and cycles spent (iii) spilling tasks to/from memory on hardware overflow; (iv) stalled on full queues; or (v) idle because there are no tasks available to run. Comparing the full-sized and $\frac{1}{16}$ caches for the same scheduler gleans the potential benefit of area dedicated to prefetching vs. caching. For example, *bfs-o* on the SPECSPAT system shows little difference in run time between a full-sized cache without prefetching (**B**) and a $\frac{1}{16}$ cache with TSP and MRS (**M**), granting an iso-performance area benefit.

Overall, TSP and MRS yield speedups as high as 3.1× (*color-r* on SPECSPAT) with gmeans up to 1.40× (SPECRAND). The main contributor is that TSP and MRS reduce non-speculative or committed cycles for every benchmark/system combination but one (*sssp-r* on NONSPECSPAT $\frac{1}{16}$ cache).

We observe increasing speedups with more prefetching. IMP [94] (**I**) performs equivalently or worse compared to no prefetching, except for *bfs-o* on NONSPECRAND. IMP’s indirect prefetching cannot trigger from task arguments, and tasks are too short for its stream prefetcher to be timely, resulting in cache pollution without additional hits. Tesseract-style prefetching (**T**) noticeably speeds up *sssp*, *bfs*, *mm*, *sc-r*, and *kcore*. Programmable TSP (**P**) produces larger speedups (except on *bfs-o*), and helps *astar* and *rbp*, but still fails to achieve a speedup on the remaining benchmarks. Trained TSP (**M**) almost always matches or outperforms programmable TSP, by as much as 3.3× (*color-r* on SPECSPAT), and achieves speedups on almost all benchmarks. Small-cache systems with TSP and MRS are often faster than full-sized baselines, showing how the baseline systems failed to utilize their caches effectively, but TSP and MRS can make even a tiny cache useful. This result implies that TSP and MRS will become more valuable as inputs grow larger.

TSP and MRS give consistent speedups on *color*, *mis*, *mm*, *sc*, *kcore*, and *rbp*. *rbp* and *mm* feature multi-object tasks [38] that struggle to achieve cache locality and require multiple addresses to be prefetched to run quickly. TSP and MRS supply these tasks with the data they need and schedule tasks to use it well. In contrast, *color*, *mis*, *sc*, and *kcore* feature many tiny tasks that only make one memory access and are sensitive to cache locality.

sssp, *bfs*, and *astar* usually see strong speedups because TSP and MRS accelerate their edgelist traversals. However *sssp-r* observes a slowdown specifically on the NONSPECSPAT $\frac{1}{16}$ -cache configuration, despite achieving high speedups with other configurations. This outlier is caused by work inefficiency. These benchmarks are sensitive to priority drift [78, 96], and MRS’s prioritization of already-cached vertices causes priority inversions. Consequently, *sssp-r* requires far more tasks to complete with TSP and MRS compared to the baseline. Each individual task runs 1.5× faster than the baseline, but there are 2.1× more of them, resulting in a 1.3× slowdown, some of which comes from queue management overhead. To achieve consistently high performance for *sssp*, *bfs*, and *astar* in a non-speculative system, an ideal scheduler would be aware of and limit priority drift. We leave applying MRS to such specialized schedulers as future work. *sssp-o* also loses its speedup when MRS is added to the SPECRAND $\frac{1}{16}$ -cache configuration. Our breakdown shows that the cause is due to speculation stalls. By deprioritizing earlier-ordered tasks that would have memory stalls, MRS slows down task commits, causing the entire system to stall when its speculative state storage is exhausted.

kmeans sees a small but consistent speedup regardless of cache size. *kmeans* is compute-heavy and mostly accesses a small set of cluster centers which fit in even the $\frac{1}{16}$ caches. Consequently, TSP and MRS only contribute a few additional cache hits.

msf-o sees a slowdown from MRS due to increased aborts. *msf* features large union-find tasks that access many memory addresses, making them vulnerable to aborts caused by data hazards [38], and MRS exacerbates the abort problem by delaying high-priority tasks. *msf-o* commits slightly fewer cycles with MRS, but suffers more aborted cycles, resulting in degraded performance overall.

The performance of *cf* is dominated by synchronization, with up to 30% of cycles spent waiting at barriers for non-speculative systems, and up to 50% of cycles spent performing work that aborts on speculative systems [40]. TSP and MRS sometimes produce a small improvement in performance, but more often have no impact.

Overall, our results glean four key insights:

Prefetching may be more important than a large cache. On many benchmarks, prefetching with a small cache is better than not prefetching with a large cache. On the SPECSPAT system, using TSP and MRS with a $\frac{1}{16}$ cache is faster than a full-sized cache for 13 of 22 benchmarks, and equivalent to the large cache baseline for another four. TSP’s additional PQ is not cheap, but it appears to be a more efficient use of area than a large cache.

Online training is essential. Trained TSP almost always outperforms programmed TSP, only suffering in pathologies where aggressive prefetching is harmful. This result is somewhat surprising, as one might expect explicitly programming the prefetch table to produce an optimal result. However, trained TSP is able to find

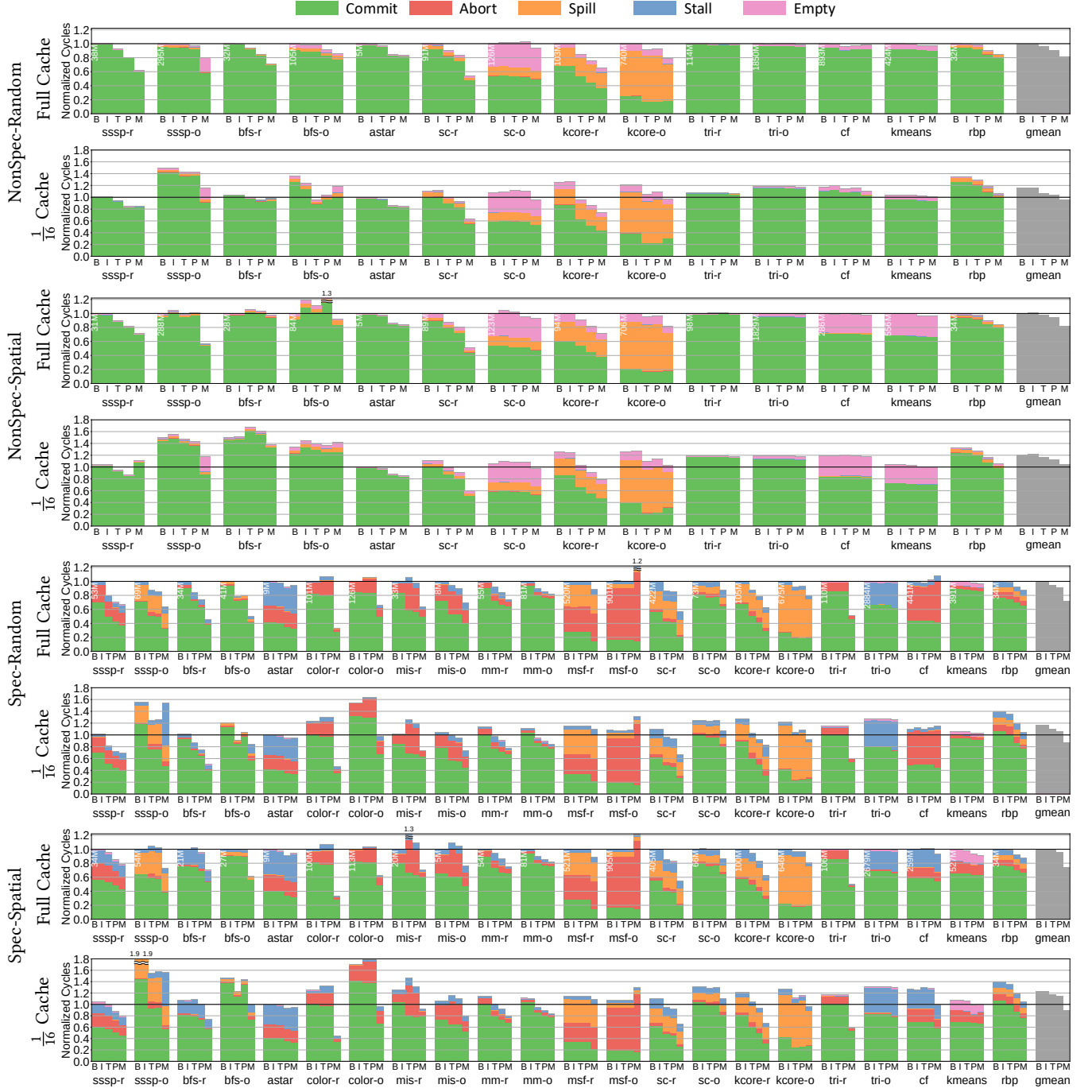


Figure 6: Normalized cycle breakdown for all systems with both full- and $\frac{1}{16}$ -sized caches. Each bar height is run time normalized to no prefetching with full-sized cache. Lower is better, 1 is no difference.

two classes of prefetches which could not be programmed into TSP. These prefetches come from (i) libraries and (ii) graph structure.

Libraries: Our benchmarks use a library to generate enqueue trees [93] that fill the system with work. Trained TSP prefetches the memory accesses of the internal and leaf nodes of these trees because it

is agnostic to the origins of the function pointers for which it is prefetching. Programmed TSP is unable to do the same because this library is common across benchmarks and accesses application data structures. Due to this reuse, it is impossible for the library to contain programmed prefetches without restricting its ABL. The

library’s internal functions are not called directly by application code, meaning that the application cannot program prefetches for the library. Consequently internal library functions cannot benefit from programmed TSP, but can benefit from trained TSP.

Graph structure: An enqueue-er-tree task accesses different data depending on the generated tree structure, which is a function of how many new tasks the tree is enqueueing. This makes programming prefetches for them statically impossible, since the access pattern is data-dependent. However, real graphs tend to have some regularity in their structures. For instance, our road graphs and power-law graphs comprise mostly low-degree vertices which will have single-node trees. Trained TSP learns this pattern and correctly prefetches the edge data it needs for most vertices, achieving higher performance than any static analysis could have provided.

Prefetching is especially valuable for difficult inputs. Across our benchmarks with two inputs, we consistently observe higher speedups from prefetching on USA-roads (-r) compared to com-Orkut (-o). USA-roads is a road network with almost all vertices having degree 4, while com-Orkut is a social media graph following a power-law distribution. These graph structures impact prefetching effectiveness in two ways.

Temporal locality: Each USA-road vertex tends to be accessed only a few times during processing, whereas com-Orkut has a few high-degree vertices that account for a large portion of vertex data accesses. The latter temporal locality implies fewer cache misses to vertex data, and therefore a lesser benefit from prefetching them.

Task hardware utilization: In many benchmarks, a task that processes a vertex could then spawn children tasks for all of the vertex’s neighbors. For USA-roads this causes a small task fanout of 4. For com-Orkut the fanout could be many thousands, saturating the hardware structures for queuing tasks or holding speculative state. This is especially notable for speculative systems, where tasks must commit in priority order but may not be enqueued in order [39]. When storage for speculative state fills up, most cores stall until the highest-priority tasks can commit to release these resources (e.g., *astar*). When MRS delays such a critical high-priority task until its prefetches fill, more cores are more likely to stall. Even worse, if MRS delays a critical task for data that it never ends up accessing, then cores stall for no reason.

Both effects make prefetching less useful on power-law graphs compared to road graphs. Prior work has found that software task scheduling structures such as OBIM [60] struggle with road graph inputs [92]. These inputs challenge software systems due to their high diameter, resulting in a wide range of priorities and long dependence chains, motivating speculative hardware to extract parallelism [39]. It is on these difficult inputs that prefetching is most effective, amplifying the advantage of task-parallel hardware.

Aggressive prefetching can make a bad scheduler worse. Some schedulers suffer from pathologies that degrade the performance of benchmarks that are a poor match for them. Aggressive prefetching can aggravate these pathologies, causing performance degradation even as it accelerates the tasks themselves. For example, prior work observed that the SPECRAND scheduler suffers from task aborts, especially on benchmarks with many writes to shared data [38]. Prefetching exacerbates this problem on *msf* in particular and *cf* to a lesser extent. These two see their tasks accelerated (fewer

committed cycles) but the increase in aborted tasks due to prefetch accesses and priority inversions overwhelms this benefit, resulting in reduced performance overall. Another aggravatable pathology is poor work efficiency. Non-speculative schedulers can require many more tasks to complete execution compared to sequential execution [27], wasting work due to priority inversions. Prioritizing ready tasks causes priority inversions that can further degrade work-efficiency. For *sssp-r* on NONSPECSPAT $\frac{1}{16}$ cache, this causes some tiles’ cores to run far ahead of others in priority order and flood the system with redundant work that forces the other tiles to spend time spilling their queues to memory, which allows the culpable tiles to run even further ahead. For both of these pathologies, switching to a better scheduler would speed up the benchmark, even without prefetching. Prefetching can make bad schedulers worse, and we leave solving this to future work.

5.3 Accuracy and coverage of TSP

We measure the accuracy and coverage of TSP on our 256-core SPECSPAT $\frac{1}{16}$ -cache system with MRS enabled, to magnify the impact of inaccurate prefetches. Fig. 7 shows the additional DRAM accesses and bandwidth (BW) utilization caused by TSP and MRS. Ten benchmarks incur no additional DRAM accesses, indicating that addresses prefetched by TSP will typically be accessed soon and that TSP is not evicting useful data from the LLC. Twelve benchmarks see increased DRAM accesses, with *bfs-o*, *color-o*, *msf-o*, *sc-o*, and *tri-r* observing over 50% increased memory traffic. In addition, other benchmarks observe increased LLC pressure with up to 20% (*mis-o*) additional NoC traffic driven by misses from the L2s to the LLC (not shown due to space constraints), but no additional accesses to DRAM. Fig. 1b showed that that *sc-r* would (over) saturate DRAM BW with *ideal* prefetching on a full-cache system. The $2\times$ BW increase in Fig. 7 with TSP and MRS bring the $\frac{1}{16}$ -cache system closer to BW saturation. *bfs-o* and *tri-r* observe $> 4\times$ increases in DRAM BW, and *tri-r* does fully utilize BW on the $\frac{1}{16}$ -cache system, but it is the only benchmark to do so. With lower baseline bandwidths, the other benchmarks remain far from saturating memory bandwidth.

The cause of these useless prefetches is the presence of moot tasks [69] in our benchmarks. Moot tasks exit early (within a few instructions) without performing most of their potential work. For example, the SSSPTask instances that return at line 2 of Listing 1

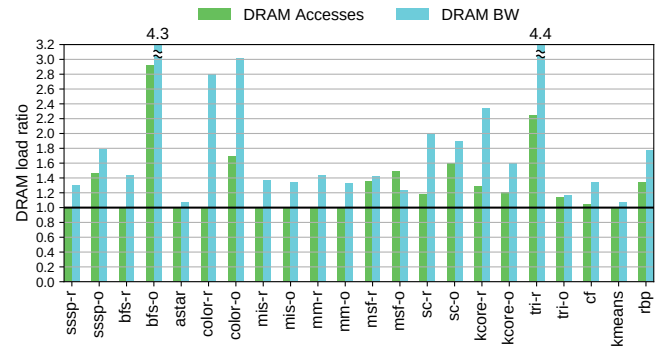


Figure 7: Additional DRAM accesses/BW caused by TSP & MRS, normalized to the SPECSPAT $\frac{1}{16}$ -cache baseline (B).

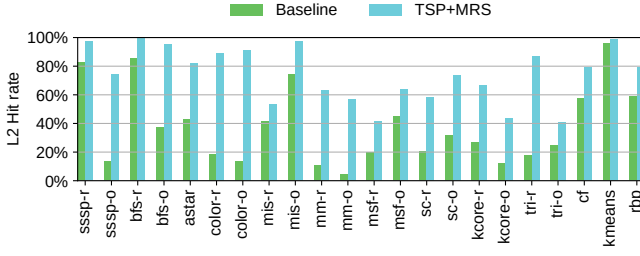


Figure 8: L2 hit rates in SPECSPAT $\frac{1}{16}$ -cache with and without TSP & MRS.

are moot. TSP aggressively prefetches the entire access pattern of a task without regard to control flow—it issues useless prefetches for moot tasks. This is why we observe extra memory accesses on the com-Orkut benchmarks (94% moot tasks in sssp-o) for sssp and bfs, but not the USA-road variant (52% moot tasks in sssp-r). We could avoid these useless prefetches by penalizing PT rows for instructions that are not accessed during task execution. However, we found that this usually hurts performance when DRAM and NoC bandwidth are not saturated. Alternatively, switching to the Hive task scheduler [69] would eliminate this problem entirely, as Hive clears out moot tasks without executing them.

Only 12 benchmarks observe increases in DRAM accesses but all benchmarks observe increased DRAM BW. This is because TSP and MRS reduce execution time and BW has time in its denominator. The benchmarks can perform the same set of memory accesses faster, increasing BW. DRAM BW utilization is up to 4.4× higher with TSP and MRS. Fig. 1b showed that only sc-r and kcore-r used more than 25% of the available DRAM BW, with most benchmarks using 10% or less. Even with this increase, the benchmarks (except tri-r) still do not saturate DRAM BW, and TSP and MRS unlock more of the performance potential of task-parallel hardware.

Fig. 8 shows that TSP and MRS significantly increase the L2 hit rate compared to the SPECSPAT $\frac{1}{16}$ -cache baseline, approaching 100% for sssp-r, bfs, mis-o, and kmeans. TSP predicts all memory accesses for these benchmarks. For others like rbp, TSP predicts many memory accesses, halving the L2 miss rate, but some accesses continue to miss in the L2. We leave the latter access patterns to future work.

5.4 Contribution of architectural features

Fig. 9 shows the performance improvement as each feature of TSP and MRS is added to the SPECSPAT $\frac{1}{16}$ -cache baseline. All benchmarks except astar, msf-o, and tri-o benefit from argument prefetching. Inner-loop prefetching slightly improves sssp-o, bfs-r, color-o, and mis-o edgelist traversals and significantly improves cf’s feature reads and kcore-r’s edgelist reads, but otherwise has little impact. The limited impact of loop prefetching is caused by short loops iterating within a single cache line—argument prefetching delivers the bulk of the benefit due to spatial locality. Completing TSP with indirect prefetching produces as large an improvement as argument prefetching does for sssp, astar, color, tri-r, and rbp, and smaller or no improvement for others. This is also the point where aggressive prefetching starts to degrade sssp-o due to over-prefetching of moot tasks (Sec. 5.3). The varied impact depends on

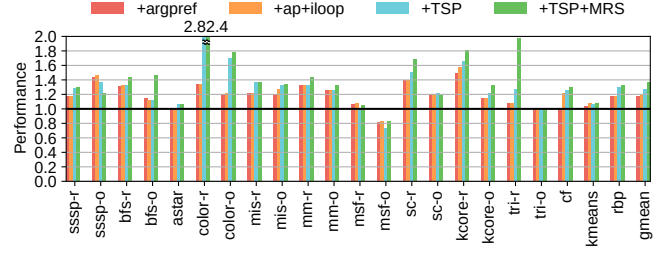


Figure 9: Performance when adding TSP and MRS features the SPECSPAT $\frac{1}{16}$ -cache baseline.

(i) whether the benchmark has dependent memory accesses and (ii) how likely those accesses are to miss in the cache. Finally, MRS’s scheduling policy provides noticeable performance improvement for bfs, color, mm, sc-r, tri-r, and less or none for others. mm and color contain both predictable and unpredictable accesses. MRS schedules tasks whose predictable accesses have filled, and when a core stalls on unpredictable accesses, this buys time for data to arrive for idle-but-prefetched tasks. bfs and tri have many tasks of equal priority (unordered [68]), allowing MRS to reorder many tasks without causing priority inversions.

5.5 Sensitivity studies

We calibrate TSP’s number of PSHRs and entries per PT row by sweeping them on the 256-core SPECSPAT $\frac{1}{16}$ -cache system with MRS enabled. Fig. 10 shows the performance effect of varying TSP’s per-tile PSHR count, normalized to 64 PSHRs. rbp demands the highest PSHR count (64) across the benchmarks, so that it can consistently prefetch ahead of its queued tasks. Therefore, we design TSP with 64 PSHRs per tile. Some benchmarks (sssp-o, color-o, sc-o and kcore-o) show peaks at a smaller number of PSHRs. Further investigation found that their performance degrades with more aggressive prefetching due to insufficient storage for speculative state. MRS adjusts the task schedule based on prefetch readiness so that tasks spend less time on cores, but these tasks are later aborted to free up speculative state for higher-priority tasks whose data was not ready. We defer balancing of schedule reordering, data readiness, and speculation resources to future work.

Fig. 11 shows the performance effect of varying the number of entries per PT row, normalized to 8. TSP requires only 1 entry per PT row to eventually learn the dependence of a single access, and 2 for an inner-loop access. More entries lets TSP learn more quickly, shortening the training phase and accelerating a larger portion of

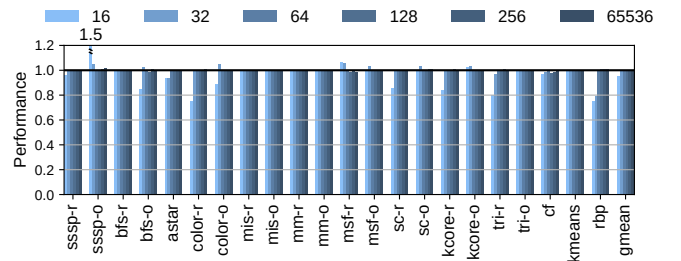


Figure 10: Sensitivity of TSP and MRS to PSHR count.

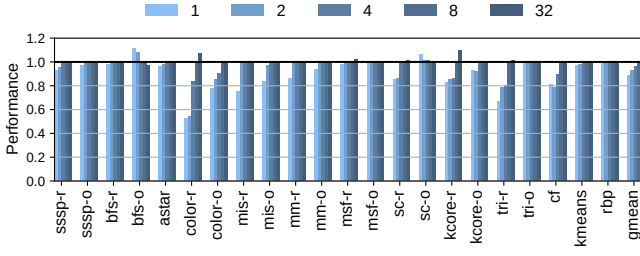


Figure 11: Sensitivity of TSP and MRS to row entry count.

execution time. This is especially true for tasks with many loads, where there are many possible dependences to test. Performance improvements are meager beyond 8 entries per row, so we size TSP PT rows at 8 entries.

5.6 Energy consumption of TSP and MRS

The TSP PT and MRS PQ require CAM lookups which occur twice per task. On average, each tile performs these accesses from every 56 cycles for SPECSPAT (0.3W for 64 tiles at 2GHz) to every 142 cycles for NONSPECSPAT $\frac{1}{16}$ -cache (0.1W). Individual benchmarks' rates depend on the average task length. They range from every 949 cycles in kmeans on NONSPECSPAT $\frac{1}{16}$ -cache to every 4 cycles in mis-o on SPECSPAT, where the average task length is 31 cycles.

6 Related Work

Hardware task-parallel systems leverage hardware task queues to extract parallelism from irregular applications. Early systems like Carbon [44] and Task Superscalar [29] relied on centralized task schedulers. In contrast, more recent systems such as Swarm [39], HD-CPS [78], and Dalorex [63] distribute their task queues across the tiles of a manycore. These distributed queues are more scalable, but suffer from priority drift [78] when using non-speculative synchronization. These hardware task-parallel systems either do not use prefetchers or prefetch only within a running task.

ADM [76], Minnow [95], DTS [89], MAPLE [62], and Hammerblade [21] do not implement hardware task schedulers, but add hardware features such as scratchpads or offload engines that improve the performance of software task-parallel runtimes. Software running on these systems may use their features to implement prefetching between tasks, but cannot react to prefetch fills.

Data prefetchers predict the future memory accesses of a program by detecting an access pattern or pre-executing address-generating code. Stride prefetchers [12] predict the common access pattern of iterating over the elements of an array, using only the address stream. Content-directed prefetchers [24] use loaded values to predict future loads, e.g., by dereferencing likely pointers. IMP [94] and its successor DMP [30] are content-directed prefetchers that are triggered by a stream prefetcher. The Dependence-Based prefetcher [74] is a content-directed prefetcher that builds a model of a program's data structures and prefetches pointer-chasing traversals early. Our TSP is a content-directed prefetcher that is triggered by the task scheduler seeding it with a task descriptor.

Runahead execution approaches [55, 57] reclaim execution resources that would otherwise be idle due to memory latencies, and use these resources to speculatively execute past stalling loads

and issue future loads early. Decoupled Vector Runahead [58] is a runahead prefetcher triggered by a stride prefetcher. Runahead execution provides an avenue for future work to expand TSP's coverage to more complex address generation and control flow.

Other approaches to prefetching include prioritizing address-generating instructions [18, 49], combining multiple prefetchers [64], reproducing past access patterns [13, 20], making the prefetcher programmable [7, 82] and providing a prefetch instruction for the programmer [87]. Some data prefetchers rely on compiler annotations to mark candidates for prefetching [31, 49]. Compiler passes can sidestep a lack of hardware prefetcher by inserting prefetch instructions into the code [6], possibly with the aid of profiling [10].

Some software task-parallel runtimes prefetch explicitly declared data during scheduling, optionally with hardware support. The Runtime-Assisted Block Prefetcher [33] is similar to programmable TSP without indirection. It modifies a software task-parallel runtime to control a hardware prefetcher. TD-NUCA [17] places data in specifically nearby LLC banks. TSP uses hardware support for task parallelism, enabling finer-grained tasks, and does not rely on programmer annotations. In addition, TSP supports indirect access patterns and MRS adjusts the task schedule as prefetches fill.

Prefetching for hardware task-parallel systems depends exclusively on programmer assistance. Tesseract [5] introduces *message-triggered prefetching*, where non-blocking RPC calls are augmented with one memory address to prefetch before the RPC runs. Message-triggered prefetching is also implemented by AB-NDP [86].

The programmable prefetch engines of EBP [67], Minnow [95], and Maple [62] require programmers to write prefetch code to run in advance of their tasks and fetch all their data. This resembles the inspector-executor model [75], where the programmer organizes their code to place all loads before all uses of the loads to maximize MLP. TSP is the first prefetcher to target hardware task-parallel systems without requiring programmer assistance.

7 Conclusion

We have demonstrated that manycores with hardware-assisted task scheduling stand to benefit from prefetching, and these prefetchers must be integrated with their task schedulers. We have also demonstrated that prefetcher-aware task schedulers achieve improved performance over prefetcher-agnostic schedulers, and that online training produces a greater speedup than static programming. We have introduced TSP and MRS, a simple prefetcher and task scheduling policy which improve the performance of already highly scalable applications by up to 3.1 $\times$. To achieve high performance, every future task-parallel system should integrate a prefetcher with its scheduler—they are better together.

Acknowledgments

We sincerely thank Natalie Enright Jerger, Steven Hill, Andreas Moshovos, Aster Plotnik, Guozheng Zhang, and the anonymous reviewers for their helpful feedback. This work was supported in part by the Digital Research Alliance of Canada, the University of Toronto Connaught Fund, Fujitsu, NSERC, and the Ontario Graduate Scholarship program.

References

- [1] 2006. 9th DIMACS Implementation Challenge: Shortest Paths.
- [2] 2015. OpenStreetMap. <https://www.openstreetmap.org>
- [3] Maleen Abeydeera and Daniel Sanchez. 2020. Chronos: Efficient Speculative Parallelism for Accelerators. In *Proc. of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS-XXV)*. ACM, 1247–1262. doi:10.1145/3373376.3378454
- [4] Maleen Abeydeera, Suvinay Subramanian, Mark C Jeffrey, Joel Emer, and Daniel Sanchez. 2017. SAM: Optimizing multithreaded cores for speculative parallelism. In *Proc. of the International Conference on Parallel Architectures and Compilation Techniques (PACT)*. IEEE/ACM. doi:10.1109/PACT.2017.37
- [5] Junwhan Ahn, Sungpack Hong, Sungjoo Yoo, Onur Mutlu, and Kiyoungh Choi. 2015. A Scalable Processing-in-Memory Accelerator for Parallel Graph Processing. In *Proc. of the International Symposium on Computer Architecture (ISCA-42)*. ACM/IEEE. doi:10.1145/2749469.275038
- [6] Sam Ainsworth and Timothy M. Jones. 2017. Software prefetching for indirect memory accesses. In *Proc. of the International Symposium on Code Generation and Optimization (CGO)*. IEEE/ACM, 305–317. doi:10.1109/CGO.2017.7863749
- [7] Sam Ainsworth and Timothy M. Jones. 2018. An Event-Triggered Programmable Prefetcher for Irregular Workloads. In *Proc. of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS-XXIII)*. ACM. doi:10.1145/3173162.3173189
- [8] Vitalii Aksenov, Dan Alistarh, and Janne H. Korhonen. 2020. Scalable Belief Propagation via Relaxed Scheduling. In *Proc. of the International Conference on Neural Information Processing Systems (NeurIPS)*. MIT Press, 22361–22372.
- [9] Dan Alistarh, Justin Kopinsky, Jerry Li, and Giorgi Nadiradze. 2017. The Power of Choice in Priority Scheduling. In *Proc. of the Symposium on Principles of Distributed Computing (PODC)*. ACM, 283–292. doi:10.1145/3087801.3087810
- [10] Grant Ayers, Heiner Litz, Christos Kozyrakis, and Parthasarathy Ranganathan. 2020. Classifying Memory Access Patterns for Prefetching. In *Proc. of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS-XXV)*. ACM, 513–526. doi:10.1145/3373376.3378498
- [11] Eduard Ayguade, Nawal Couet, Alejandro Duran, Jay Hoeflinger, Yuan Lin, Federico Massaioli, Xavier Teruel, Priya Unnikrishnan, and Guansong Zhang. 2009. The Design of OpenMP Tasks. *IEEE Transactions on Parallel and Distributed Systems* 20, 3 (2009), 404–418. doi:10.1109/TPDS.2008.105
- [12] Jean-Loup Baer and Tien-Fu Chen. 1991. An effective on-chip preloading scheme to reduce data access penalty. In *Proc. of the Conference on Supercomputing (SC)*. ACM/IEEE, 176–186. doi:10.1145/125826.125932
- [13] Mohammad Bakhshalipour, Mehran Shakerinava, Pejman Lotfi-Kamran, and Hamid Sarbazi-Azad. 2019. Bingo Spatial Data Prefetcher. In *Proc. of the International Symposium on High Performance Computer Architecture (HPCA-25)*. IEEE. doi:10.1109/HPCA.2019.00053
- [14] Vignesh Balaji, Neal Crago, Aamer Jaleel, and Brandon Lucia. 2021. P-OPT: Practical Optimal Cache Replacement for Graph Analytics. In *Proc. of the International Symposium on High Performance Computer Architecture (HPCA-27)*. IEEE, 668–681. doi:10.1109/HPCA51647.2021.00062
- [15] Scott Beamer, Krste Asanovic, and David Patterson. 2015. Locality Exists in Graph Processing: Workload Characterization on an Ivy Bridge Server. In *Proc. of the International Symposium on Workload Characterization (IISWC)*. IEEE, 56–65. doi:10.1109/IISWC.2015.12
- [16] Robert D. Blumofe, Christopher F. Joerg, Bradley C. Kuszmaul, Charles E. Leiserson, Keith H. Randall, and Yuli Zhou. 1995. Cilk: An Efficient Multithreaded Runtime System. In *Proc. of the Symposium on Principles and Practice of Parallel Programming (PPoPP)*. ACM. doi:10.1145/209936.209958
- [17] Paul Caheny, Lluc Alvarez, Marc Casas, and Miquel Moreto. 2022. TD-NUCA: runtime driven management of NUCA caches in task dataflow programming models. In *Proc. of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*. IEEE/ACM, 1–15. doi:10.1109/SC41404.2022.00085
- [18] Trevor E Carlson, Wim Heirman, Osman Allam, Stefanos Kaxiras, and Lieven Eeckhout. 2015. The load slice core microarchitecture. In *Proc. of the International Symposium on Computer Architecture (ISCA-42)*. ACM/IEEE. doi:10.1145/2749469.2750407
- [19] J. Lawrence Carter and Mark Wegman. 1979. Universal classes of hash functions. *J. Comput. System Sci.* 18, 2 (1979), 143–154. doi:10.1016/0022-0000(79)90044-8
- [20] Zixiao Chen, Chentao Wu, Yunfei Gu, Ranhao Jia, Jie Li, and Minyi Guo. 2025. Gaze into the Pattern: Characterizing Spatial Patterns with Internal Temporal Correlations for Hardware Prefetching. In *Proc. of the International Symposium on High Performance Computer Architecture (HPCA-31)*. IEEE, 173–187. doi:10.1109/HPCA61900.2025.00024
- [21] Lin Cheng, Max Ruttenberg, Dai Cheol Jung, Dustin Richmond, Michael Taylor, Mark Oskin, and Christopher Batten. 2023. Beyond Static Parallel Loops: Supporting Dynamic Task Parallelism on Manycore Architectures with Software-Managed Scratchpad Memories. In *Proc. of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS-XXVIII)*. ACM. doi:10.1145/3582016.3582020
- [22] George Chrysos. 2012. Intel Xeon Phi coprocessor (codename Knights Corner). In *Proc. of the Hot Chips Symposium (HCS)*. IEEE. doi:10.1109/HOTCHIPS.2012.7476487
- [23] G. Contreras and M. Martonosi. 2008. Characterizing and improving the performance of Intel Threading Building Blocks. In *Proc. of the International Symposium on Workload Characterization (IISWC)*. IEEE. doi:10.1109/IISWC.2008.4636091
- [24] Robert Cooksey, Stephan Jourdan, and Dirk Grunwald. 2002. A stateless, content-directed data prefetching mechanism. In *Proc. of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS-X)*. ACM. doi:10.1145/605397.605427
- [25] Vidushi Dadu, Sihao Liu, and Tony Nowatzki. 2021. PolyGraph: Exposing the Value of Flexibility for Graph Processing Accelerators. In *Proc. of the International Symposium on Computer Architecture (ISCA-48)*. ACM/IEEE, 595–608. doi:10.1109/ISCA52012.2021.00053
- [26] John D Davis, James Laudon, and Kunle Olukotun. 2005. Maximizing CMP throughput with mediocre cores. In *Proc. of the International Conference on Parallel Architectures and Compilation Techniques (PACT)*. IEEE/ACM. doi:10.1109/PACT.2005.42
- [27] Laxman Dhulipala, Guy Blelloch, and Julian Shun. 2017. Julianne: A Framework for Parallel Graph Algorithms Using Work-efficient Bucketing. In *Proc. of the Symposium on Parallelism in Algorithms and Architectures (SPAA)*. ACM, 293–304. doi:10.1145/3087556.3087580
- [28] Fares Elsabbagh, Shabnam Sheikhha, Victor A. Ying, Quan M. Nguyen, Joel S. Emer, and Daniel Sanchez. 2023. Accelerating RTL Simulation with Hardware-Software Co-Design. In *Proc. of the International Symposium on Microarchitecture (MICRO-56)*. IEEE/ACM, 153–166. doi:10.1145/3613424.3614257
- [29] Y. Etsion, F. Cabarcas, A. Rico, A. Ramirez, R.M. Badia, E. Ayguade, J. Labarta, and M. Valero. 2010. Task Superscalar: An Out-of-Order Task Pipeline. In *Proc. of the International Symposium on Microarchitecture (MICRO-43)*. IEEE/ACM, 89–100. doi:10.1109/MICRO.2010.13
- [30] Gelin Fu, Tian Xia, Zhongpei Luo, Ruiyang Chen, Wenzhe Zhao, and Pengju Ren. 2024. Differential-Matching Prefetcher for Indirect Memory Access. In *Proc. of the International Symposium on High Performance Computer Architecture (HPCA-30)*. IEEE, 439–453. doi:10.1109/HPCA57654.2024.00040
- [31] Adi Fuchs, Shie Mannor, Uri Weiser, and Yoav Etsion. 2014. Loop-Aware Memory Prefetching Using Code Block Working Sets. In *Proc. of the International Symposium on Microarchitecture (MICRO-47)*. IEEE/ACM, 533–544. doi:10.1109/MICRO.2014.27
- [32] Chao Gao, Mahbod Afarin, Shafiu Rahman, Nael Abu-Ghazaleh, and Rajiv Gupta. 2023. MEGA Evolving Graph Accelerator. In *Proc. of the International Symposium on Microarchitecture (MICRO-56)*. IEEE/ACM, 310–323. doi:10.1145/3613424.3614260
- [33] Victor Garcia, Alejandro Rico, Carlos Villavieja, Paul Carpenter, Nacho Navarro, and Alex Ramirez. 2017. Adaptive runtime-assisted block prefetching on chip-multiprocessors. *International journal of parallel programming* 45 (2017), 530–550. doi:10.1007/s10766-016-0431-8
- [34] Tae Jun Ham, Lisa Wu, Narayanan Sundaram, Nadathur Satish, and Margaret Martonosi. 2016. Graphiconado: A High-Performance and Energy-Efficient Accelerator for Graph Analytics. In *Proc. of the International Symposium on Microarchitecture (MICRO-49)*. IEEE/ACM. doi:10.1109/MICRO.2016.7783759
- [35] F. Maxwell Harper and Joseph A. Konstan. 2015. The MovieLens Datasets: History and Context. *ACM Transactions on Interactive Intelligent Systems (TiiS)* 5, 4 (2015). doi:10.1145/2827872
- [36] William Hasenplaugh, Tim Kaler, Tao B. Schardl, and Charles E. Leiserson. 2014. Ordering heuristics for parallel graph coloring. In *Proc. of the Symposium on Parallelism in Algorithms and Architectures (SPAA)*. ACM, 166–177. doi:10.1145/2612669.2612697
- [37] Muhammad Amber Hassaan, Donald Nguyen, and Keshav Pingali. 2015. Kinetic Dependence Graphs. In *Proc. of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS-XX)*. ACM, 457–471. doi:10.1145/2775054.2694363
- [38] Mark C. Jeffrey, Suvinay Subramanian, Maleen Abeydeera, Joel Emer, and Daniel Sanchez. 2016. Data-centric execution of speculative parallel programs. In *Proc. of the International Symposium on Microarchitecture (MICRO-49)*. IEEE/ACM, 1–13. doi:10.1109/MICRO.2016.7783708
- [39] Mark C. Jeffrey, Suvinay Subramanian, Cong Yan, Joel Emer, and Daniel Sanchez. 2015. A scalable architecture for ordered parallelism. In *Proc. of the International Symposium on Microarchitecture (MICRO-48)*. IEEE/ACM, 228–241. doi:10.1145/2830772.2830777
- [40] Mark C. Jeffrey, Victor A. Ying, Suvinay Subramanian, Hyun Ryong Lee, Joel Emer, and Daniel Sanchez. 2018. Harmonizing speculative and non-speculative execution in architectures for ordered parallelism. In *Proc. of the International Symposium on Microarchitecture (MICRO-51)*. IEEE/ACM, 217–230. doi:10.1109/MICRO.2018.00026
- [41] Shizhi Jiang, Qiusong Yang, and Yiwei Ci. 2022. Merging Similar Patterns for Hardware Prefetching. In *Proc. of the International Symposium on Microarchitecture (MICRO-55)*. IEEE/ACM, 1012–1026. doi:10.1109/MICRO56248.2022.00071

- [42] Changkyu Kim, Doug Burger, and Stephen W. Keckler. 2002. An adaptive, non-uniform cache structure for wire-delay dominated on-chip caches. In *Proc. of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS-X)*. ACM, 211–222. doi:10.1145/605397.605420
- [43] Ji Kim, Shunning Jiang, Christopher Torng, Moyang Wang, Shreesha Srinath, Berkin Ilbeyi, Khalid Al-Hawaj, and Christopher Batten. 2017. Using Intra-Core Loop-Task Accelerators to Improve the Productivity and Performance of Task-Based Parallel Programs. In *Proc. of the International Symposium on Microarchitecture (MICRO-50)*. IEEE/ACM, 759–773. doi:10.1145/3123939.3136952
- [44] Sanjeev Kumar, Christopher Hughes, and Anthony Nguyen. 2007. Carbon: architectural support for fine-grained parallelism on chip multiprocessors. In *Proc. of the International Symposium on Computer Architecture (ISCA-34)*. ACM/IEEE. doi:10.1145/1273440.1250683
- [45] Jaemin Lee, Changhee Jung, Daeseob Lim, and Yan Solihin. 2008. Prefetching with helper threads for loosely coupled multiprocessor systems. *IEEE Transactions on Parallel and Distributed Systems* 20, 9 (2008), 1309–1324. doi:10.1109/TPDS.2008.224
- [46] Daan Leijen, Wolfram Schulte, and Sebastian Burckhardt. 2009. The Design of a Task Parallel Library. In *Proc. of the International Conference on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA)*. ACM, 227–242. doi:10.1145/1640089.1640106
- [47] Charles Leiserson and Tao Scharld. 2010. A work-efficient parallel breadth-first search algorithm. In *Proc. of the Symposium on Parallelism in Algorithms and Architectures (SPAA)*. ACM, 303–314. doi:10.1145/1810479.1810534
- [48] Jure Leskovec and Andrej Krevl. 2014. SNAP datasets: Stanford large network dataset collection. <http://snap.stanford.edu/data>.
- [49] Heiner Litz, Grant Ayers, and Parthasarathy Ranganathan. 2022. CRISP: critical slice prefetching. In *Proc. of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS-XXVII)*. ACM. doi:10.1145/3503222.3507745
- [50] Jinshu Liu, Hamid Hadian, Yuyue Wang, Daniel S. Berger, Marie Nguyen, Xun Jian, Sam H. Noh, and Huaicheng Li. 2025. Systematic CXL Memory Characterization and Performance Analysis at Scale. In *Proc. of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 1203–1217. doi:10.1145/3676641.3715987
- [51] Chi-Keung Luk, Robert Cohn, Robert Muth, Harish Patil, Artur Klauser, Geoff Lowney, Steven Wallace, Vijay Janapa Reddi, and Kim Hazelwood. 2005. Pin: Building customized program analysis tools with dynamic instrumentation. In *Proc. of the Conference on Programming Language Design and Implementation (PLDI)*. ACM, 190–200. doi:10.1145/1064978.1065034
- [52] Chi Cao Minh, Jaewoong Chung, Christos Kozyrakis, and Kunle Olukotun. 2008. STAMP: Stanford Transactional Applications for Multi-Processing. In *Proc. of the International Symposium on Workload Characterization (IISWC)*. IEEE. doi:10.1109/IISWC.2008.4636089
- [53] Lucas Morais, Vitor Silva, Alfredo Goldman, Carlos Alvarez, Jaime Bosch, Michael Frank, and Guido Araujo. 2019. Adding Tightly-Integrated Task Scheduling Acceleration to a RISC-V Multi-core Processor. In *Proc. of the International Symposium on Microarchitecture (MICRO-52)*. IEEE/ACM, 861–872. doi:10.1145/3352460.3358271
- [54] Anurag Mukkara, Nathan Beckmann, Maleen Abeydeera, Xiaosong Ma, and Daniel Sanchez. 2018. Exploiting locality in graph analytics through hardware-accelerated traversal scheduling. In *Proc. of the International Symposium on Microarchitecture (MICRO-51)*. IEEE/ACM. doi:10.1109/MICRO.2018.00010
- [55] O. Mutlu, J. Stark, C. Wilkerson, and Y.N. Patt. 2003. Runahead execution: an alternative to very large instruction windows for out-of-order processors. In *Proc. of the International Symposium on High Performance Computer Architecture (HPCA-9)*. IEEE. doi:10.1109/HPCA.2003.1183532
- [56] Ajeya Naithani, Sam Ainsworth, Timothy M. Jones, and Lieven Eeckhout. 2021. Vector Runahead. In *Proc. of the International Symposium on Computer Architecture (ISCA-48)*. ACM/IEEE. doi:10.1109/ISCA52012.2021.00024
- [57] Ajeya Naithani, Josué Feliu, Almutaz Adileh, and Lieven Eeckhout. 2020. Precise Runahead Execution. In *Proc. of the International Symposium on High Performance Computer Architecture (HPCA-26)*. IEEE, 397–410. doi:10.1109/HPCA47549.2020.00040
- [58] Ajeya Naithani, Jaime Roelands, Sam Ainsworth, Timothy M. Jones, and Lieven Eeckhout. 2023. Decoupled Vector Runahead. In *Proc. of the International Symposium on Microarchitecture (MICRO-56)*. IEEE/ACM. doi:10.1145/3613424.3614255
- [59] Agustín Navarro-Torres, Biswabandan Panda, Jesús Alastruey-Benedé, Pablo Ibáñez, Víctor Viñals-Yúfera, and Alberto Ros. 2022. Berti: an Accurate Local-Delta Data Prefetcher. In *Proc. of the International Symposium on Microarchitecture (MICRO-55)*. IEEE/ACM, 975–991. doi:10.1109/MICRO56248.2022.00072
- [60] Donald Nguyen, Andrew Lenharth, and Keshav Pingali. 2013. A Lightweight Infrastructure for Graph Analytics. In *Proc. of the Symposium on Operating Systems Principles (SOSP-24)*. ACM, 456–471. doi:10.1145/2517349.2522739
- [61] K. Nii, T. Amano, N. Watanabe, M. Yamawaki, K. Yoshinaga, M. Wada, and I. Hayashi. 2014. A 28nm 400MHz 4-Parallel 1.6Gsearch/s 80Mb Ternary CAM. In *Proc. of the International Solid-State Circuits Conference (ISSCC)*. IEEE. doi:10.1109/ISSCC.2014.6757417
- [62] Marcelo Orenes-Vera, Aninda Manocha, Jonathan Balkind, Fei Gao, Juan L. Aragón, David Wentzlaff, and Margaret Martonosi. 2022. Tiny but mighty: designing and realizing scalable latency tolerance for manycore SoCs. In *Proc. of the International Symposium on Computer Architecture (ISCA-49)*. ACM/IEEE. doi:10.1145/3470496.3527400
- [63] Marcelo Orenes-Vera, Esin Tureci, David Wentzlaff, and Margaret Martonosi. 2023. Dalorex: A data-local program execution and architecture for memory-bound applications. In *Proc. of the International Symposium on High Performance Computer Architecture (HPCA-29)*. IEEE, 718–730. doi:10.1109/HPCA56546.2023.10071089
- [64] Samuel Pakalapati and Biswabandan Panda. 2020. Bouquet of Instruction Pointers: Instruction Pointer Classifier-based Spatial Hardware Prefetching. In *Proc. of the International Symposium on Computer Architecture (ISCA-47)*. ACM/IEEE. doi:10.1109/ISCA45697.2020.00021
- [65] Heidi Pan, Krste Asanović, Robert Cohn, and Chi-Keung Luk. 2005. Controlling program execution through binary instrumentation. *SIGARCH Comput. Archit. News* 33, 5 (2005), 45–50. doi:10.1145/1127577.1127587
- [66] Biswabandan Panda. 2023. CLIP: Load Criticality based Data Prefetching for Bandwidth-constrained Many-core Systems. In *Proc. of the International Symposium on Microarchitecture (MICRO-56)*. IEEE/ACM. doi:10.1145/3613424.3614245
- [67] Vassilis Papaefstathiou, Manolis G.H. Katevenis, Dimitrios S. Nikolopoulos, and Dionisios Pnevmatikatos. 2013. Prefetching and cache management using task lifetimes. In *Proc. of the International Conference on Supercomputing (ICS)*. ACM, 325–334. doi:10.1145/2464996.2465443
- [68] Keshav Pingali, Donald Nguyen, Milind Kulkarni, Martin Burtscher, M. Amber Hassaan, Rashid Kaleem, Tsung-Hsien Lee, Andrew Lenharth, Roman Manevich, Mario Méndez-Lojo, Dimitrios Prountzos, and Xin Sui. 2011. The tao of parallelism in algorithms. In *Proc. of the Conference on Programming Language Design and Implementation (PLDI)*. ACM, 12–25. doi:10.1145/1993498.1993501
- [69] Gilead Posluns, Yan Zhu, Guowei Zhang, and Mark C. Jeffrey. 2022. A Scalable Architecture for Reprioritizing Ordered Parallelism. In *Proceedings of the 49th Annual International Symposium on Computer Architecture (ISCA-49)*. ACM/IEEE, 437–453. doi:10.1145/3470496.3527387
- [70] Shafiu Rahman, Nael Abu-Ghazaleh, and Rajiv Gupta. 2020. GraphPulse: An Event-Driven Hardware Accelerator for Asynchronous Graph Processing. In *Proc. of the International Symposium on Microarchitecture (MICRO-53)*. IEEE/ACM. doi:10.1109/MICRO50266.2020.00078
- [71] Shafiu Rahman, Mahbod Afarin, Nael Abu-Ghazaleh, and Rajiv Gupta. 2021. JetStream: Graph Analytics on Streaming Data with Event-Driven Hardware Accelerator. In *Proc. of the International Symposium on Microarchitecture (MICRO-54)*. IEEE/ACM, 1091–1105. doi:10.1145/3466752.3480126
- [72] James Reinders. 2007. *Intel Threading Building Blocks*. O'Reilly & Associates.
- [73] Aditya Rohan, Biswabandan Panda, and Prakhara Agarwal. 2020. Reverse Engineering the Stream Prefetcher for Profit. In *Proc. European Symposium on Security and Privacy Workshops (EuroS&PW)*. IEEE. doi:10.1109/EuroSPW51379.2020.00098
- [74] Amir Roth, Andreas Moshovos, and Gurindar S. Sohi. 1998. Dependence based prefetching for linked data structures. In *Proc. of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS-VIII)*. ACM. doi:10.1145/291069.291034
- [75] Joel H. Salz, Ravi Mirchandaney, and Kay Crowley. 1991. Run-Time Parallelization and Scheduling of Loops. *IEEE Trans. Comput.* 40, 5 (1991).
- [76] Daniel Sanchez, Richard Yoo, and Christos Kozyrakis. 2010. Flexible Architectural Support for Fine-Grain Scheduling. In *Proc. of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS-XV)*. ACM. doi:10.1145/1736020.1736055
- [77] Tao B. Scharld, William S. Moses, and Charles E. Leiserson. 2017. Tapir: Embedding Fork-Join Parallelism into LLVM's Intermediate Representation. In *Proc. of the Symposium on Principles and Practice of Parallel Programming (PPoPP)*. ACM. doi:10.1145/3018743.3018758
- [78] Mohsin Shan and Omer Khan. 2022. HD-CPS: Hardware-assisted Drift-aware Concurrent Priority Scheduler for Shared Memory Multicores. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 528–542. doi:10.1109/HPCA53966.2022.00046
- [79] Julian Shun and Guy E. Blelloch. 2013. Ligma: a lightweight graph processing framework for shared memory. In *Proc. of the Symposium on Principles and Practice of Parallel Programming (PPoPP)*. ACM. doi:10.1145/2442516.2442530
- [80] Julian Shun, Guy E. Blelloch, Jeremy T. Fineman, Phillip B. Gibbons, Aapo Kyrola, Harsha Vardhan Simhadri, and Kanat Tangwongsan. 2012. Brief announcement: The Problem Based Benchmark Suite. In *Proc. of the Symposium on Parallelism in Algorithms and Architectures (SPAA)*. ACM, 68–70. doi:10.1145/2312005.2312018
- [81] William J. Starke, Brian W. Thompson, Jeff A. Stuecheli, and José E. Moreira. 2021. IBM's POWER10 Processor. *IEEE Micro* 41, 2 (2021), 7–14. doi:10.1109/MM.2021.3058632
- [82] Nishil Talati, Kyle May, Armand Behrooz, Yichen Yang, Kuba Kaszyk, Christos Vasiladiotis, Tarunesh Verma, Lu Li, Brandon Nguyen, Jiawen Sun, John Magnus Morton, Agreen Ahmadi, Todd Austin, Michael O'Boyle, Scott Mahlke, Trevor Mudge, and Ronald Dreslinski. 2021. Prodigy: Improving the Memory Latency

- of Data-Indirect Irregular Workloads Using Hardware-Software Co-Design. In *Proc. of the International Symposium on High Performance Computer Architecture (HPCA-27)*. IEEE. doi:10.1109/HPCA51647.2021.00061
- [83] Xubin Tan, Jaume Bosch, Miquel Vidal, Carlos Álvarez, Daniel Jiménez-González, Eduard Ayguadé, and Mateo Valero. 2017. General Purpose Task-Dependence Management Hardware for Task-Based Dataflow Programming Models. In *Proc. of the International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 244–253. doi:10.1109/IPDPS.2017.48
- [84] J. M. Tendler, J. S. Dodson, J. S. Fields, H. Le, and B. Sinharoy. 2002. POWER4 system microarchitecture. *IBM Journal of Research and Development* 46, 1 (2002), 5–25. doi:10.1147/rd.461.0005
- [85] Shyamkumar Thoziyoor, Naveen Muralimanohar, Jung Ho Ahn, and Norman P. Jouppi. 2008. *CACTI 5.1*. Technical Report HPL-2008-20. HP Labs.
- [86] Boyu Tian, Qihang Chen, and Mingyu Gao. 2023. ABNDP: Co-optimizing Data Access and Load Balance in Near-Data Processing. In *Proc. of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS-XXVIII)*. ACM. doi:10.1145/3582016.3582026
- [87] Steven P. Vanderwiel and David J. Lilja. 2000. Data prefetch mechanisms. *Comput. Surveys* 32, 2 (2000). doi:10.1145/358923.358939
- [88] Jose Rodrigo Sanchez Vicarte, Michael Flanders, Riccardo Paccagnella, Grant Garrett-Grossman, Adam Morrison, Christopher W. Fletcher, and David Kohlbrenner. 2022. Augury: Using Data Memory-Dependent Prefetchers to Leak Data at Rest. In *Proc. of the Symposium on Security and Privacy (SP)*. IEEE, 1491–1505. doi:10.1109/SP46214.2022.9833570
- [89] Moyang Wang, Tuan Ta, Lin Cheng, and Christopher Batten. 2020. Efficiently supporting dynamic task parallelism on heterogeneous cache-coherent systems. In *Proc. of the International Symposium on Computer Architecture (ISCA-47)*. ACM/IEEE, 173–186. doi:10.1109/ISCA45697.2020.00025
- [90] David Wentzlaff, Patrick Griffin, Henry Hoffmann, Liewei Bao, Bruce Edwards, Carl Ramey, Matthew Mattina, Chyi-Chang Miao, John F. Brown III, and Anant Agarwal. 2007. On-chip interconnection architecture of the Tile Processor. *IEEE Micro* 27, 5 (2007), 15–31. doi:10.1109/MM.2007.4378780
- [91] Jaewon Yang and Jure Leskovec. 2012. Defining and Evaluating Network Communities Based on Ground-Truth. In *Proc. International Conference on Data Mining (ICDM)*. IEEE, 745–754. doi:10.1109/ICDM.2012.138
- [92] Serif Yesil, Azin Heidarshenas, Adam Morrison, and Josep Torrellas. 2019. Understanding Priority-Based Scheduling of Graph Algorithms on a Shared-Memory Platform. In *Proc. of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC19)*. ACM, 1–14. doi:10.1145/3295500.3356160
- [93] Victor A. Ying, Mark C. Jeffrey, and Daniel Sanchez. 2020. T4: Compiling sequential code for effective speculative parallelization in hardware. In *Proc. of the International Symposium on Computer Architecture (ISCA-47)*. ACM/IEEE, 159–172. doi:10.1109/ISCA45697.2020.00024
- [94] Xiangyao Yu, Christopher J. Hughes, Nadathur Satish, and Srinivas Devadas. 2015. IMP: Indirect Memory Prefetcher. In *Proc. of the International Symposium on Microarchitecture (MICRO-48)*. IEEE/ACM. doi:10.1145/2830772.2830807
- [95] Dan Zhang, Xiaoyu Ma, Michael Thomson, and Derek Chiou. 2018. Minnow: Lightweight Offload Engines for Worklist Management and Worklist-Directed Prefetching. In *Proc. of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS-XXIII)*. ACM. doi:10.1145/3173162.3173197
- [96] Guozheng Zhang, Gilead Posluns, and Mark C. Jeffrey. 2024. Multi Bucket Queues: Efficient Concurrent Priority Scheduling. In *Proc. of the Symposium on Parallelism in Algorithms and Architectures (SPAA)*. ACM, 113–124. doi:10.1145/3626183.3659962